

1	Reminder	1	14.3	Burnside's Lemma	25
1.1	Observations and Tricks	1	15	Special Numbers	25
1.2	Bug List	1	15.1	Fibonacci Series	25
2	Init (Linux)	1	15.2	Prime Numbers	25
2.1	vimrc	1	15.3	Number of Divisors	25
2.2	bashrc	1	15.4	Distinct Prime Factors	25
3	Basic	1			
3.1	Template (Using Codebook)	1			
3.2	PBDS, Random	1			
3.3	Debug	2			
3.4	SVG Writer	2			
3.5	Python	2			
3.6	Stress Tests	2			
4	Data Structure	2			
4.1	Mo's Algorithm	2			
4.2	CDQ	3			
4.3	Persistent Treap	3			
4.4	Li Chao Tree	3			
4.5	Time Segment Tree	3			
5	DP	4			
5.1	SOS DP	4			
5.2	Divide and Conquer DP	4			
5.3	Dynamic DP	4			
6	Graph	4			
6.1	Max Clique	4			
6.2	Bellman-Ford	4			
6.3	System of Difference Constraints	5			
6.4	Graph Girth	5			
6.5	Euler Trail	5			
6.6	Vertex BCC (Round Square Tree)	5			
6.7	Edge BCC	6			
6.8	Kth Shortest Path	6			
6.9	SCC - Tarjan	7			
6.10	2SAT	7			
7	Tree	7			
7.1	Tree Isomorphism (Rooted Trees)	7			
7.2	Tree Isomorphism (Unrooted Trees)	7			
7.3	Heavy Light Decomposition	7			
7.4	Virtual Tree	7			
8	Matching	7			
8.1	Bipartite Matching	7			
8.2	Bipartite Weighted Matching	8			
8.3	General Matching	8			
8.4	General Weighted Matching	9			
9	Flow	10			
9.1	Flow Methods	10			
9.2	Dinic	10			
9.3	ISAP	11			
9.4	Bounded Max Flow	11			
9.5	MCMF	11			
9.6	Push-Relabel	12			
9.7	Gomory-Hu Tree	12			
9.8	Global Min Cut	12			
10	String	13			
10.1	Rolling Hash	13			
10.2	KMP, Z Value	13			
10.3	Manacher	13			
10.4	Suffix Array	13			
10.5	Suffix Automaton	13			
10.6	Minimum Rotation	14			
10.7	Aho Corasick	14			
11	Geometry	14			
11.1	Template	14			
11.2	Basic Operations	14			
11.3	Lower Concave Hull	15			
11.4	Pick's Theorem	15			
11.5	Minimum Enclosing Circle	15			
11.6	PolyUnion	15			
11.7	Minkowski Sum	15			
12	Number Theory	16			
12.1	Mod Sum	16			
12.2	Extended Lucas Theorem	16			
12.3	Prime Sieve and Defactor	17			
12.4	Harmonic Series	17			
12.5	Count Number of Divisors	17			
12.6	數論分塊	17			
12.7	Pollard's rho	17			
12.8	Miller Rabin	18			
12.9	Discrete Log	18			
12.10	Discrete Sqrt	18			
12.11	Fast Power	18			
12.12	Extend GCD	18			
12.13	Mu + Phi	18			
12.14	Other Formulas	19			
12.15	Polynomial	19			
12.16	Counting Primes	20			
12.17	Linear Sieve for Other Number Theoretic Functions	20			
13	Numerical	21			
13.1	Polynomials and recurrences	21			
13.2	Optimization	22			
13.3	Matrices	23			
13.4	Fourier transforms	24			
14	Combinatorics	25			
14.1	Catalan Number	25			
14.2	Bertrand's Ballot Theorem	25			

1 Reminder

1.1 Observations and Tricks

- Contribution Technique
- 二分圖/Spanning Tree/DFS Tree
- 行、列操作互相獨立
- 奇偶性
- 當 s, t 遞增並且 $t = f(s)$, 對 s 二分搜不好做, 可以改成對 t 二分搜, 再算 $f(t)$
- 啟發式合併
- Permutation Normalization (做一些平移對齊兩個 permutation)
- 枚舉 $a_1 \sim a_n$ 再枚舉 $a_n \sim a_1$ 可以包在一個迴圈
- 兩個凸型函數相加還是凸型函數, 相減不一定
- 一個區間的 $\text{mex} = k$, 表示這個區間包含 $0 \sim k-1$ 所有數字, 並且「 U - 區間」的最小值 $= k$ 。

1.2 Bug List

- 沒開 long long
- 陣列截出界/陣列開不夠大
- 寫好的函式 (例如 `init()`、`build()`) 忘記呼叫
- 0-base / 1-base
- 線段樹改值懶標初始值不能設為 0
- DFS 的時候不小心覆寫到全域變數
- 多筆測資不能沒讀完直接 return
- vector 超級肥, 小 vector 請用 array, 例如矩陣快速冪

2 Init (Linux)

2.1 vimrc

```
1syn on
2set nu rnu cul mouse=a cin et ts=2 sw=2 sts=2 autochdir
  cb=unnamedplus
3no ;
4ino {<CR> {<CR>}<Esc>ko
5ca Hash w !cpp -dD -P \ tr -d '[:space:]' \ md5sum \ cut
  -c-6
```

```
1" Gino's .vimrc "
2"no <C-h> ^ | <C-l> $ | h b | l e | b l | e h
```

2.2 bashrc

```
1gogo() {
2  g++ -std=c++20 -O2 "$1" && echo run && ./a.out
3}
4alias vig='vim -u ~/.vimrc-gino'
```

3 Basic

3.1 Template (Using Codebook)

```
1// CryptoGrapheR
2#define SZ(c) ((int)(c).size())
3// kactl
4#define sz(x) (int)(x).size()
5#define rep(i,a,b) for(int i=(a);i<(b);i++)
6using ll = long long;
7using vi = vector<int>;
8using pii = pair<int, int>;
9// ckiseki
10#define all(x) begin(x), end(x)
11#define pb emplace_back
12#define REP(i, l, r) for (int i=(l); i<=(r); ++i)
13const ll LINF = 1e18;
14template<class A, class B> bool chmin(A& a, B& b) {
15  return b < a ? a = b, 1 : 0;
16}
```

3.2 PBDS, Random

```
1#include <bits/extc++.h>
2using namespace __gnu_pbds;
3// map
4tree<int, int, less<>, rb_tree_tag,
  tree_order_statistics_node_update> tr;
5tr.order_of_key(element);
6tr.find_by_order(rank);
7// set
8tree<int, null_type, less<>, rb_tree_tag,
  tree_order_statistics_node_update> tr;
9tr.order_of_key(element);
10tr.find_by_order(rank);
11// priority queue
```

```

12 __gnu_pbds::priority_queue<int, less<int>> big_q;
13 __gnu_pbds::priority_queue<int, greater<int>> small_q;
14 q1.join(q2); // join
15
16 mt19937
17 gen(chrono::steady_clock::now().time_since_epoch().count());
18 uniform_int_distribution<int>(a, b)(rng) // [a, b]
19 uniform_real_distribution<double>(a, b)(rng) // (a, b)
20 shuffle(v.begin(), v.end(), gen);

```

3.3 Debug

```

1 #define debug(...) cerr << "\x1b[33m" # __VA_ARGS__ " : \
  \x1b[0m" \
2 [(auto&&... a){ ((cerr << a << ' '), ...); } \
  (__VA_ARGS__), cerr << '\n'
3 #define output(...) \
4 [(auto&&... a){ ((cout << a << ' '), ...); } \
  (__VA_ARGS__), cout << '\n'
5 int v = 42;
6 format("{:b}", v); // "101010"
7 format("{:#b}", v); // "0b101010"
8 format("{:x}", v); // "0x2a"
9 format("{:X}", v); // "0X2A"
10 int x = 7;
11 format("{:05d}", x); // "00007"
12 format("{:>5}", x); // " 7"
13 format("{:<5}", x); // "7 "
14 format("{:^5}", x); // " 7 "
15 format("{:*^5}", x); // "***7***"
16 double pi = 3.1415926535;
17 format("{:.2f}", pi); // "3.14"

```

3.4 SVG Writer

```

1 // Author: ckiseki
2 class SVG {
3     void p(string_view s) { o << s; }
4     void p(string_view s, auto v, auto... vs) {
5         auto i = s.find('$');
6         o << s.substr(0, i) << v, p(s.substr(i + 1, vs...));
7     }
8     ostream o; string c = "red";
9 public:
10    SVG(auto f, auto x1, auto y1, auto x2, auto y2) : o(f) {
11        p("<svg xmlns='http://www.w3.org/2000/svg' "
12          "viewBox='$ $ $ $'>\n"
13          "<style>{*stroke-width:0.5%;}</style>\n",
14          x1, -y2, x2 - x1, y2 - y1); }
15    ~SVG() { p("</svg>\n"); }
16    void color(string nc) { c = nc; }
17    void line(auto x1, auto y1, auto x2, auto y2) {
18        p("<line x1='$' y1='$' x2='$' y2='$' stroke='$' />\n",
19          x1, -y1, x2, -y2, c); }
20    void circle(auto x, auto y, auto r) {
21        p("<circle cx='$' cy='$' r='$' stroke='$' "
22          "fill='none' />\n", x, -y, r, c); }
23    void text(auto x, auto y, string s, int w = 12) {
24        p("<text x='$' y='$' font-size='$px'>$</text>\n",
25          x, -y, w, s); }
26};

1 SVG svg("out.svg", -200, -200, 200, 200);
2 svg.color("lightgray"); // ↓ drawing x-y plane
3 svg.line(-200, 0, 200, 0); svg.line(0, -200, 0, 200);
4 svg.color("blue"); svg.line(10, 10, 80, 80);
5 svg.color("red"); svg.circle(50, 50, 1);

```

3.5 Python

```

1 import sys
2 input = sys.stdin.readline
3 readStr = lambda: input().rstrip('\r\n')
4 readInt = lambda: int(input())
5 readInts = lambda: list(map(int, input().split()))
6
7 from decimal import *
8 getcontext().prec = 50 # precision
9 x = Decimal(str(x))
10 y = Decimal("6.7")
11 x *= y
12 print(x.quantize(Decimal("0.000001"),
13   rounding=ROUND_HALF_EVEN))
14 # ROUND_CEILING : To INF (2.9 -> 3, -2.1 -> -2)
15 # ROUND_FLOOR : To -INF (2.1 -> 2, -2.9 -> -3)
16 # ROUND_HALF_EVEN : To nearest even (2.5 -> 2, 3.5 -> 4)
17 # ROUND_HALF_UP : Half away from 0 (2.5 -> 3, -2.5 -> -3)
18 # ROUND_HALF_DOWN : Half towards 0 (2.5 -> 2, -2.5 -> -2)
19 # ROUND_UP : Away from 0 (2.1 -> 3, -2.1 -> -3)
20 # ROUND_DOWN : Truncate to 0 (2.9 -> 2, -2.9 -> -2)
21 # <Default>: ROUND_HALF_EVEN
22 # <Standard School Math>: ROUND_HALF_UP

```

3.6 Stress Tests

```

1 from random import *
2 n = randint(1, 10)
3 arr = list(randint(1, 20) for i in range(n))
4 print(*arr)
5 # permutation / non-repetitive sequence
6 perm = sample(range(1, n + 1), n)
7 arr = sample(range(1, 10000 + 1), n)
8 # string
9 from string import *
10 s = "".join(choices(ascii_lowercase, k=n))
11 t = "".join(choices(ascii_lowercase[:3], k=n)) # (a|b|c)*
12 u = "".join(choices(ascii_lowercase + ascii_uppercase +
13   digits, k=n))
14 # palindrome
15 half = "".join(choices(ascii_lowercase, k=(n + 1) // 2))
16 palindrome = half + (half[-2::-1] if n % 2 else half[::-1])
17 # ===== tree =====
18 n = 30
19 ## shallow tree
20 edges = [(randint(1, u - 1), u) for u in range(2, n + 1)]
21 ## deep tree
22 k = int(sqrt(n))
23 edges = [(randint(max(1, u - k), u - 1), u) for u in range(2, n
24   + 1)]
25 for e in edges:
26     print(*e)
27
28 edges = [(1, i) for i in range(2, n + 1)] ## star
29 edges = [(i, i + 1) for i in range(1, n)] ## chain
30 edges = [(i // 2, i) for i in range(2, n + 1)] ## complete
31 # ===== graph =====
32 ## not necessarily connected, no multi-edge or self-loop
33 n = 10
34 density = 0.3 # graph density (0.0 ~ 1.0)
35 edges = [(i, j) for i in range(1, n+1) for j in range(i+1,
36   n+1) if random() < density]
37 n, extra_edges = 10, 5
38 edges = set()
39 ##### ... generate spanning tree with codes above
40 while len(edges) < n - 1 + extra_edges:
41     u, v = randint(1, n), randint(1, n)
42     if u != v:
43         edges.add((min(u, v), max(u, v)))
44 edges = list(edges)
45 ## DAG
46 perm = sample(range(1, n + 1), n)
47 edges = []
48 for _ in range(m):
49     i, j = sorted(sample(range(n), 2))
50     edges.append((perm[i], perm[j]))
51 ## Bipartite Graph
52 n1, n2 = 6, 7
53 left, right = list(range(1, n1+1)), list(range(n1+1,
54   n1+n2+1))
55 edges = [(choice(left), choice(right)) for _ in range(m)]

```

```

1 g++ -std=c++20 -o buggy $1
2 g++ -std=c++20 -o ac ac.cpp
3 for i in {1..100}; do
4     echo "TEST $i"
5     python3 gen.py > in
6     ./ac < in > oa
7     ./buggy < in > ob
8     if ! diff -q oa ob; then
9         cat in <(echo "=== AC") oa <(echo "=== WA") ob
10        break
11    fi
12 done

```

4 Data Structure

4.1 Mo's Algorithm

```

1 // segments are 0-based
2 int B = max(1, (int)sqrt(n));
3 sort(Q.begin(), Q.end(), [&](const auto& a, const auto& b) {
4     if (a[0]/B != b[0]/B) return a[0]/B < b[0]/B;
5     return (a[0]/B & 1) ? a[1] < b[1] : a[1] > b[1];
6 });
7 ll cur = 0; // current answer
8 int pl = 0, pr = -1;
9 for (auto& qi : Q) {
10    // get (l, r, qid) from qi
11    while (pl < l) del(pl++); while (pl > l) add(--pl);
12    while (pr < r) add(++pr); while (pr > r) del(pr--);

```

```
13 ans[qid] = cur;
14 }
```

4.2 CDQ

```
1// preparation
2// 1. write a 1-base BIT
3// 2. init(n, mxc): n is number of elements, mxc = 值域
4// 3. build info array CDQ.v by yourself (x, y, z, id)
5// 4. call solve()
6// => cdq.ans[i] is number of j s.t. (xj <= xi, yj <= yi, zj
<= zi) (j != i)
7 struct CDQ {
8     struct info {
9         int x, y, z, id;
10        info(int x, int y, int z, int id):
11            x(x), y(y), z(z), id(id) {}
12        info():
13            x(0), y(0), z(0), id(0) {}
14    };
15    int n, mxc;
16    BIT bit;
17    vector<info> v;
18    vector<ll> ans;
19    void init(int _n, int _mxc) {
20        n = _n; mxc = _mxc;
21        v.assign(n, info{});
22        ans.assign(n, 0);
23    }
24    void dfs(int l, int r) {
25        if (l >= r) return;
26        int mid = (l+r) >> 1;
27        dfs(l, mid); dfs(mid+1, r);
28
29        vector<info> tmp;
30        vector<int> bit_op;
31        int pl = l, pr = mid+1;
32
33        while (pl <= mid && pr <= r) {
34            if (v[pl].y <= v[pr].y) {
35                tmp.emplace_back(v[pl]);
36                bit.upd(v[pl].z, 1);
37                bit_op.emplace_back(v[pl].z);
38                pl++;
39            }
40            else {
41                tmp.emplace_back(v[pr]);
42                ans[v[pr].id] += bit.qry(v[pr].z);
43                pr++;
44            }
45        }
46        while (pl <= mid) {
47            tmp.emplace_back(v[pl]);
48            pl++;
49        }
50        while (pr <= r) {
51            tmp.emplace_back(v[pr]);
52            ans[v[pr].id] += bit.qry(v[pr].z);
53            pr++;
54        }
55        for (int i = l, j = 0; i <= r; i++, j++) v[i] = tmp[j];
56        for (auto& op : bit_op) bit.upd(op, -1);
57    }
58    void solve() {
59        bit.init(mxc);
60        sort(v.begin(), v.end(), [&](const info& a, const info&
b){
61            return (a.x == b.x ? (a.y == b.y ? (a.z == b.z ?
a.id < b.id : a.z < b.z) : a.y < b.y) : a.x < b.x);
62        });
63        dfs(0, n-1);
64        map<pair<int, pii>, int> s;
65        sort(v.begin(), v.end(), [&](const info& a, const info&
b){
66            return a.id > b.id;
67        });
68        for (int i = 0, id = n-1; i < n; i++, id--) {
69            pair<int, pii> tmp = make_pair(v[i].x,
make_pair(v[i].y, v[i].z));
70            make_pair(v[i].y, v[i].z));
71            auto it = s.find(tmp);
72            if (it != s.end())
73                ans[id] += it->second;
74            s[tmp]++;
75        }
76    };
77 }
```

4.3 Persistent Treap

```
1// Author: Ian
2 struct node {
3     node *l, *r;
```

```
4     char c; int v, sz;
5     node(char x = '$'): c(x), v(mt()), sz(1) {
6         l = r = nullptr;
7     }
8     node(node* p) { *this = *p; }
9     void pull() {
10        sz = 1;
11        for (auto i : {l, r})
12            if (i) sz += i->sz;
13    }
14    arr[maxn], *ptr = arr;
15    inline int size(node* p) { return p ? p->sz : 0; }
16    node* merge(node* a, node* b) {
17        if (!a || !b) return a ? a : b;
18        if (a->v < b->v) {
19            node* ret = new(ptr++) node(a);
20            ret->r = merge(ret->r, b); ret->pull();
21            return ret;
22        }
23        else {
24            node* ret = new(ptr++) node(b);
25            ret->l = merge(a, ret->l); ret->pull();
26            return ret;
27        }
28    }
29    P<node*> split(node* p, int k) {
30        if (!p) return {nullptr, nullptr};
31        if (k >= size(p->l) + 1) {
32            auto [a, b] = split(p->r, k - size(p->l) - 1);
33            node* ret = new(ptr++) node(p);
34            ret->r = a; ret->pull();
35            return {ret, b};
36        }
37        else {
38            auto [a, b] = split(p->l, k);
39            node* ret = new(ptr++) node(p);
40            ret->l = b; ret->pull();
41            return {a, ret};
42        }
43    }
44 }
```

4.4 Li Chao Tree

```
1// Author: Ian
2// Function: For a set of lines L, find the maximum L_i(x)
in L in O(lg n).
3 typedef long double ld;
4 constexpr int maxn = 5e4 + 5;
5 struct line {
6     ld a, b;
7     ld operator()(ld x) { return a * x + b; }
8 } arr[(maxn + 1) << 2];
9 bool operator<(line a, line b) { return a.a < b.a; }
10 #define m ((l+r)>>1)
11 void insert(line x, int i = 1, int l = 0, int r = maxn) {
12     if (r - l == 1) {
13         if (x(l) > arr[i](l))
14             arr[i] = x;
15         return;
16     }
17     line a = max(arr[i], x), b = min(arr[i], x);
18     if (a(m) > b(m))
19         arr[i] = a, insert(b, i << 1, l, m);
20     else
21         arr[i] = b, insert(a, i << 1 | 1, m, r);
22 }
23 ld query(int x, int i = 1, int l = 0, int r = maxn) {
24     if (x < l || r <= x) return -numeric_limits<ld>::max();
25     if (r - l == 1) return arr[i](x);
26     return max({arr[i](x), query(x, i << 1, l, m), query(x, i
<< 1 | 1, m, r)});
27 }
28 #undef m
```

4.5 Time Segment Tree

```
1// Author: Ian
2 constexpr int maxn = 1e5 + 5;
3 V<P<int>> arr[(maxn + 1) << 2];
4 V<int> dsu, sz;
5 V<tuple<int, int, int>> his;
6 int cnt, q;
7 int find(int x) {
8     return x == dsu[x] ? x : find(dsu[x]);
9 }
10 inline bool merge(int x, int y) {
11     int a = find(x), b = find(y);
12     if (a == b) return false;
13     if (sz[a] > sz[b]) swap(a, b);
14     his.emplace_back(a, b, sz[b]), dsu[a] = b, sz[b] +=
sz[a];
```

```

15     return true;
16 };
17 inline void undo() {
18     auto [a, b, s] = his.back(); his.pop_back();
19     dsu[a] = a, sz[b] = s;
20 }
21 #define m ((l + r) >> 1)
22 void insert(int ql, int qr, P<int> x, int i = 1, int l = 0,
23             int r = q) {
24     // debug(ql, qr, x); return;
25     if (qr <= l || r <= ql) return;
26     if (ql <= l && r <= qr) {arr[i].push_back(x); return;}
27     if (qr <= m)
28         insert(ql, qr, x, i << 1, l, m);
29     else if (m <= ql)
30         insert(ql, qr, x, i << 1 | 1, m, r);
31     else {
32         insert(ql, qr, x, i << 1, l, m);
33         insert(ql, qr, x, i << 1 | 1, m, r);
34     }
35 }
36 void traversal(V<int>& ans, int i = 1, int l = 0, int r = q)
37 {
38     int opcnt = 0;
39     // debug(i, l, r);
40     for (auto [a, b] : arr[i])
41         if (merge(a, b))
42             opcnt++, cnt--;
43     if (r - l == 1) ans[l] = cnt;
44     else {
45         traversal(ans, i << 1, l, m);
46         traversal(ans, i << 1 | 1, m, r);
47     }
48     while (opcnt--)
49         undo(), cnt++;
50     arr[i].clear();
51 }
52 #undef m
53 inline void solve() {
54     int n, m; cin >> n >> m >> q, q++;
55     dsu.resize(cnt = n), sz.assign(n, 1);
56     iota(dsu.begin(), dsu.end(), 0);
57     // a, b, time, operation
58     unordered_map<ll, V<int>> s;
59     for (int i = 0; i < m; i++) {
60         int a, b; cin >> a >> b;
61         if (a > b) swap(a, b);
62         s[(ll)a << 32 | b].emplace_back(0);
63     }
64     for (int i = 1; i < q; i++) {
65         int op, a, b;
66         cin >> op >> a >> b;
67         if (a > b) swap(a, b);
68         switch (op) {
69             case 1:
70                 s[(ll)a << 32 | b].push_back(i);
71                 break;
72             case 2:
73                 auto tmp = s[(ll)a << 32 | b].back();
74                 s[(ll)a << 32 | b].pop_back();
75                 insert(tmp, i, P<int> {a, b});
76         }
77     }
78     for (auto [p, v] : s) {
79         int a = p >> 32, b = p & -1;
80         while (v.size()) {
81             insert(v.back(), q, P<int> {a, b});
82             v.pop_back();
83         }
84     }
85     V<int> ans(q);
86     traversal(ans);
87     for (auto i : ans)
88         cout << i << ' ';
89     cout << endl;
90 }

```

5 DP

5.1 SOS DP

```

1 // Author: Gino
2 // Function: Solve problems that enumerates subsets of
3 // subsets (3^n => n*2^n)
4 for (int msk = 0; msk < (1<<n); msk++) {
5     for (int i = 1; i <= n; i++) {
6         if (msk & (1<<(i - 1))) {
7             dp[msk][i] = dp[msk][i - 1] + dp[msk ^ (1<<(i - 1))][i - 1];
8         } else {
9             dp[msk][i] = dp[msk][i - 1];
10        }
11    }
12 }

```

```

9 } } }

```

5.2 Divide and Conquer DP

```

1 /* Author: Simon Lindholm (License: CC0)
2 * Description: Given $a[i] = \min_{lo(i) \le k < hi(i)} (f(i, k))$ where the (minimal)
3 * optimal $k$ increases with $i$, computes $a[i]$ for $i = L..R-1$.
4 * Time: $O((N + (hi-lo)) \log N)$ */
5 struct DP { // Modify at will:
6     int lo(int ind) { return 0; }
7     int hi(int ind) { return ind; }
8     ll f(int ind, int k) { return dp[ind][k]; }
9     void store(int ind, int k, ll v) { res[ind] = pii(k, v); }
10 }
11 void rec(int L, int R, int LO, int HI) {
12     if (L >= R) return;
13     int mid = (L + R) >> 1;
14     pair<ll, int> best(LLONG_MAX, LO);
15     rep(k, max(LO, lo(mid)), min(HI, hi(mid)))
16         best = min(best, make_pair(f(mid, k), k));
17     store(mid, best.second, best.first);
18     rec(L, mid, LO, best.second+1);
19     rec(mid+1, R, best.second, HI);
20 }
21 void solve(int L, int R) { rec(L, R, INT_MIN, INT_MAX); }
22 };

```

5.3 Dynamic DP

- 適用情境： $dp_i = M_i \cdot dp_{i-1} \Rightarrow dp_i = M_i M_{i-1} \cdots M_1 dp_0$
- 當 M_i 需要動態修改，且 M_i 是廣義矩陣乘法：

$$C_{ij} = \bigoplus_k A_{ik} \otimes B_{kj}$$

- 滿足 $(\oplus, \otimes)$ 是半環 ($\otimes$ 對 $\oplus$ 有分配律、 $\otimes$ 和 $\oplus$ 有結合律、 $\oplus$ 有交換律)
- 此時便可用線段樹維護 $M_n M_{n-1} \cdots M_1$ 的乘積。線段樹上的 pos 是 M_{pos} ，但 pull 時要 $\text{st}[\text{idx}] = \text{mul}(\text{st}[\text{cr}], \text{st}[\text{cl}])$

6 Graph

6.1 Max Clique

```

1 // max N = 64
2 typedef unsigned long long ll;
3 struct MaxClique {
4     ll nb[ N ], n, ans;
5     void init( ll _n ) {
6         n = _n;
7         for ( int i = 0 ; i < n ; i ++ ) nb[ i ] = 0LLU;
8     }
9     void add_edge( ll _u , ll _v ) {
10         nb[ _u ] |= ( 1LLU << _v );
11         nb[ _v ] |= ( 1LLU << _u );
12     }
13     void B( ll r , ll p , ll x , ll cnt , ll res ) {
14         if ( cnt + res < ans ) return;
15         if ( p == 0LLU && x == 0LLU ) {
16             if ( cnt > ans ) ans = cnt;
17             return;
18         }
19         ll y = p | x; y &= -y;
20         ll q = p & ( ~nb[ int( log2( y ) ) ] );
21         while ( q ) {
22             ll i = int( log2( q & (-q) ) );
23             B( r | ( 1LLU << i ) , p & nb[ i ] , x & nb[ i ] ,
24               cnt + 1LLU , __builtin_popcountll( p &
25               nb[ i ] ) );
26             q &= ~( 1LLU << i );
27             p &= ~( 1LLU << i );
28             x |= ( 1LLU << i );
29         }
30     }
31     int solve() {
32         ans = 0;
33         ll _set = 0;
34         if ( n < 64 ) _set = ( 1LLU << n ) - 1;
35         else {
36             for ( ll i = 0 ; i < n ; i ++ ) _set |= ( 1LLU << i );
37         }
38         B( 0LLU , _set , 0LLU , 0LLU , n );
39         return ans;
40     }
41 };

```

6.2 Bellman-Ford

```

1 // Author: Gino
2 // n, m = #(vertices), #(edges), and vertices are 1-based
3 // 1. BellmanFord bf(G, n, m);

```



```

4// 2-1: bf.calcDis(s);
5// => bf.dis[u] := shortest dis from s to u
6// => bf.dis[u] := LINF (s can't reach u)
7// => bf.dis[u] := -LINF (dis[u] can be arbitrary small)
8// 2-2: bf.findNegCycle();
9// => return false (if no neg cycle)
10// => return true (has neg cycle, and gives an example:
   bf.neg_cycle)
11// Time: O(VE), Space: O(V + E)
12const ll LINF = 4e18;
13struct BellmanFord {
14    const vector<vector<pair<int, ll>>& G;
15    int n, m; // #(vertices), #(edges)
16    BellmanFord(const auto& G, int n, int m):
17        G(G), n(n), m(m) {}
18
19    vector<ll> dis;
20    vector<int> nth_relax, pa;
21    void run_bf(const auto& src) {
22        dis.assign(n + 1, LINF);
23        nth_relax.assign(n + 1, 0);
24        pa.assign(n + 1, -1);
25        for (auto& s : src) dis[s] = 0;
26        for (int rlx = 1; rlx <= n; rlx++) {
27            for (int u = 1; u <= n; u++) {
28                if (dis[u] == LINF) continue; // !important
29                for (auto& [v, w] : G[u]) {
30                    if (dis[v] > dis[u] + w) {
31                        dis[v] = dis[u] + w;
32                        pa[v] = u;
33                        if (rlx == n) nth_relax[v] = 1;
34                    } } } } // 5 brackets
35    void calcDis(int s) {
36        run_bf(vector<int>{s});
37        queue<int> q;
38        for (int u = 1; u <= n; u++)
39            if (nth_relax[u]) q.push(u);
40        while (!q.empty()) {
41            int u = q.front(); q.pop();
42            for (auto& [v, w] : G[u]) {
43                if (!nth_relax[v]) {
44                    nth_relax[v] = 1;
45                    q.push(v);
46                } } }
47        for (int u = 1; u <= n; u++)
48            if (nth_relax[u]) dis[u] = -LINF;
49    }
50    vector<int> neg_cycle;
51    bool findNegCycle() {
52        auto src = views::iota(1, n + 1);
53        run_bf(src);
54        auto it = ranges::find_if(src, [&](int s){ return
nth_relax[s]; });
55        if (it == src.end()) return false;
56        int ptr = *it;
57        for (int i = 0; i < n; i++) ptr = pa[ptr];
58
59        neg_cycle.clear();
60        int cur = ptr;
61        while (true) {
62            neg_cycle.emplace_back(cur);
63            if (cur == ptr && neg_cycle.size() > 1) break;
64            cur = pa[cur];
65        }
66        ranges::reverse(neg_cycle);
67        return true;
68    }
69};

```

6.3 System of Difference Constraints

```

1 vector<vector<pair<int, ll>>> G;
2 void add(int u, int v, ll w) { G[u].push_back({v, w}); }

```

- $x_u - x_v \leq c \Rightarrow \text{add}(v, u, c)$
- $x_u - x_v \geq c \Rightarrow \text{add}(u, v, -c)$
- $x_u - x_v = c \Rightarrow \text{add}(v, u, c), \text{add}(u, v, -c)$
- $x_u \geq c \Rightarrow \text{add super vertex } x_0 = 0, \text{ then } x_u - x_0 \geq c \Rightarrow \text{add}(u, 0, -c)$
- Don't forget non-negative constraints for every variable if specified implicitly.
- Interval sum $\Rightarrow$ Use prefix sum to transform into differential constraints. Don't forget $S_{i+1} - S_i \geq 0$ if x_i needs to be non-negative.
- $x_u/x_v \leq c \Rightarrow \log x_u - \log x_v \leq \log c$

6.4 Graph Girth

Run BFS for every node, when encountered non-BFS-tree edge, update answer (min cycle length) with $\text{dis}[u] + \text{dis}[v] + 1$. Time $O(VE)$.

6.5 Euler Trail

```

1// Author: kactl (Simon Lindholm), wrapped by Gino
2// Vertices are 1-based, Edge ID are 0-based
3// 1. EulerWalk euler(n, true(directed) /
   false(undirected));
4// 2. euler.addEdge(u, v);
5// 3. int result = euler.solve();
6// => -1: nothing (euler.path, euler.path_edges = {} )
7// => 1: Euler trail,
8// => 2: Euler circuit
9//     euler.path = {u_1, ..., u_{m+1}}
10//     euler.path_edges = {e_1, ..., e_m}
11// Time: O(V + E), Space: O(V + E)
12// Test: V, E <= 2e5, 99ms
13struct EulerWalk {
14    int n, m = 0; bool dir;
15    vector<vector<pii>> G; vi deg;
16    EulerWalk(int n, bool dir) : n(n), dir(dir), G(n + 1),
deg(n + 1) {}
17    void addEdge(int u, int v) {
18        G[u].push_back({v, m});
19        if (!dir) G[v].push_back({u, m});
20        deg[u]++, deg[v] += dir ? -1 : 1;
21        m++;
22    }
23    vi path, path_edges;
24    void walk(int src) {
25        path.clear(); path_edges.clear();
26        vi its(n + 1), visE(m);
27        function<void(int, int)> dfs = [&](int u, int e_in) {
28            while (its[u] < sz(G[u])) {
29                auto [v, e] = G[u][its[u]++];
30                if (visE[e]) continue;
31                visE[e] = 1;
32                dfs(v, e);
33            }
34            path.push_back(u);
35            if (e_in != -1) path_edges.push_back(e_in);
36        };
37        dfs(src, -1);
38        if (sz(path) != m + 1) {
39            path.clear(); path_edges.clear(); return;
40        }
41        ranges::reverse(path); ranges::reverse(path_edges);
42    }
43    int solve() {
44        int s = -1, odd = 0, src = 0, dst = 0;
45        for (int u = 1; u <= n; u++) {
46            if (!dir && (deg[u] & 1)) odd++, s = u;
47            if (dir && deg[u] == 1) s = u, src++;
48            if (dir && deg[u] == -1) dst++;
49            if (dir && abs(deg[u]) > 1) return -1;
50        }
51        if (!dir && odd != 0 && odd != 2) return -1;
52        if (dir && !((src == 0 && dst == 0) || (src == 1 && dst == 1))) return -1;
53        if (s == -1) for (int u = 1; u <= n; u++) if (!
G[u].empty()) { s = u; break; }
54        if (s == -1) s = 1;
55        walk(s); return path.empty() ? -1 : path.front() ==
path.back() ? 2 : 1;
56    }
57};

```

6.6 Vertex BCC (Round Square Tree)

```

1// Author: std_abs
2struct BCC { // 1-based, remember to build
3    int n, nbcc; // note for isolated point
4    vector<vector<int>> g, _g; // id > n: bcc
5    vector<int> pa, dep, low, stk, pa2, dep2;
6    void dfs(int v, int p) {
7        dep[v] = low[v] = ~p ? dep[p] + 1 : 0;
8        stk.pb(v), pa[v] = p;
9        for (int u : g[v]) if (u != p) {
10            if (low[u] == -1) {
11                dfs(u, v), low[v] = min(low[v], low[u]);
12                if (low[u] >= dep[v]) {
13                    int id = nbcc++, x;
14                    do {
15                        x = stk.back(), stk.pop_back();
16                        _g[id + n + 1].pb(x), _g[x].pb(id + n + 1);
17                    } while (x != u);
18                    _g[id + n + 1].pb(v), _g[v].pb(id + n + 1);
19                }

```

```

20     } else low[v] = min(low[v], dep[u]);
21 }
22 }
23 bool is_cut(int x) { return sz(_g[x]) != 1; }
24 vector<int> bcc(int id) { return _g[id + n + 1]; }
25 int bcc_id(int u, int v) {
26     return pa2[dep2[u] < dep2[v] ? v : u] - n - 1; }
27 void dfs2(int v, int p) {
28     dep2[v] = ~p ? dep2[p] + 1 : 0; pa2[v] = p;
29     for (int u : _g[v]) if (u != p) dfs2(u, v);
30 }
31 void build() {
32     low.assign(n + 1, -1);
33     for (int i = 1; i <= n; ++i) if (low[i] == -1)
34         dfs(i, -1), dfs2(i, -1);
35 }
36 void add_edge(int u, int v) {
37     g[u].pb(v), g[v].pb(u); }
38 BCC (int _n) : n(_n), nbcc(0), g(n + 1), _g(2 * n + 1),
39     pa(n + 1), dep(n + 1), low(n + 1), stk(), pa2(2 * n +
40     1), dep2(2 * n + 1) {}
41 };

```

6.7 Edge BCC

```

1 // Author: std_abs
2 // Vertices 1-based
3 // Edge ID 0-based, BCCID 0-based
4 // 1. EBCC ebcc(n); // n = #(vertices)
5 // 2. ebcc.add_edge(u, v);
6 // 3. ebcc.build();
7 // => nbcc: number of EBCC
8 // => bccs: stores all edge bccs
9 // => bcc_id[u], is_bridge[Edge ID]
10 struct EBCC { // 1-based, remember to build
11     int n, m, nbcc;
12     vector<vector<pair<int, int>>> G;
13     vector<vector<int>> bccs;
14     vector<int> pa, low, dep, bcc_id, stk, is_bridge;
15     void dfs(int v, int p, int f) {
16         low[v] = dep[v] = ~p ? dep[p] + 1 : 0;
17         stk.push_back(v), pa[v] = p;
18         for (auto [u, e] : G[v]) {
19             if (low[u] == -1)
20                 dfs(u, v, e), low[v] = min(low[v], low[u]);
21             else if (e != f)
22                 low[v] = min(low[v], dep[u]);
23         }
24         if (low[v] == dep[v]) {
25             if (~f) is_bridge[f] = true;
26             bccs.push_back({});
27             int id = nbcc++, x;
28             do {
29                 x = stk.back(), stk.pop_back();
30                 bcc_id[x] = id;
31                 bccs[id].emplace_back(x);
32             } while (x != v);
33         }
34     }
35     void build() {
36         is_bridge.assign(m, 0);
37         for (int i = 1; i <= n; ++i) if (low[i] == -1)
38             dfs(i, -1, -1);
39     }
40     void add_edge(int u, int v) {
41         G[u].emplace_back(v, m), G[v].emplace_back(u, m++);
42     }
43     EBCC (int _n) : n(_n), m(0), nbcc(0), G(n + 1), pa(n + 1),
44         low(n + 1, -1), dep(n + 1), bcc_id(n + 1), stk() {}
45 };

```

6.8 Kth Shortest Path

```

1 // time: O(|E| lg |E| + |V| lg |V| + K)
2 // memory: O(|E| lg |E| + |V|)
3 struct KSP { // 1-base
4     struct nd {
5         int u, v; ll d;
6         nd(int ui=0, int vi=0, ll di=INF) { u=ui; v=vi; d=di; }
7     };
8     struct heap { nd* edge; int dep; heap* chd[4]; };
9     static int cmp(heap* a, heap* b)
10     { return a->edge->d > b->edge->d; }
11     struct node {
12         int v; ll d; heap* H; nd* E;
13         node() {}
14         node(ll _d, int _v, nd* _E) { d=_d; v=_v; E=_E; }
15         node(heap* _H, ll _d) { H=_H; d=_d; }
16         friend bool operator<(node a, node b)
17         { return a.d > b.d; }

```

```

18     };
19     int n, k, s, t, dst[N]; nd *nxt[N];
20     vector<nd*> g[N], rg[N]; heap *nullNd, *head[N];
21     void init(int _n, int _k, int _s, int _t) {
22         n=_n; k=_k; s=_s; t=_t;
23         for (int i=1; i<=n; i++) {
24             g[i].clear(); rg[i].clear();
25             nxt[i]=NULL; head[i]=NULL; dst[i]=-1;
26         }
27     }
28     void addEdge(int ui, int vi, ll di) {
29         nd* e=new nd(ui, vi, di);
30         g[ui].push_back(e); rg[vi].push_back(e);
31     }
32     queue<int> dfsQ;
33     void dijkstra() {
34         while (dfsQ.size()) dfsQ.pop();
35         priority_queue<node> Q; Q.push(node(0, t, NULL));
36         while (!Q.empty()) {
37             node p=Q.top(); Q.pop(); if (dst[p.v] != -1) continue;
38             dst[p.v]=p.d; nxt[p.v]=p.E; dfsQ.push(p.v);
39             for (auto e: rg[p.v]) Q.push(node(p.d+e->d, e->u, e));
40         }
41     }
42     heap* merge(heap* curNd, heap* newNd) {
43         if (curNd==nullNd) return newNd;
44         heap* root=new heap; memcpy(root, curNd, sizeof(heap));
45         if (newNd->edge->d < curNd->edge->d) {
46             root->edge=newNd->edge;
47             root->chd[2]=newNd->chd[2];
48             root->chd[3]=newNd->chd[3];
49             newNd->edge=curNd->edge;
50             newNd->chd[2]=curNd->chd[2];
51             newNd->chd[3]=curNd->chd[3];
52         }
53         if (root->chd[0]->dep < root->chd[1]->dep)
54             root->chd[0]=merge(root->chd[0], newNd);
55         else root->chd[1]=merge(root->chd[1], newNd);
56         root->dep=max(root->chd[0]->dep,
57             root->chd[1]->dep)+1;
58         return root;
59     }
60     vector<heap*> V;
61     void build() {
62         nullNd=new heap; nullNd->dep=0; nullNd->edge=new nd;
63         fill(nullNd->chd, nullNd->chd+4, nullNd);
64         while (not dfsQ.empty()) {
65             int u=dfsQ.front(); dfsQ.pop();
66             if (!nxt[u]) head[u]=nullNd;
67             else head[u]=head[nxt[u]->v];
68             V.clear();
69             for (auto&& e: g[u]) {
70                 int v=e->v;
71                 if (dst[v] == -1) continue;
72                 e->d+=dst[v]-dst[u];
73                 if (nxt[u] != e) {
74                     heap* p=new heap; fill(p->chd, p->chd+4, nullNd);
75                     p->dep=1; p->edge=e; V.push_back(p);
76                 }
77             }
78             if (V.empty()) continue;
79             make_heap(V.begin(), V.end(), cmp);
80             #define L(X) ((X<<1)+1)
81             #define R(X) ((X<<1)+2)
82             for (size_t i=0; i<V.size(); i++) {
83                 if (L(i) < V.size()) V[i]->chd[2]=V[L(i)];
84                 else V[i]->chd[2]=nullNd;
85                 if (R(i) < V.size()) V[i]->chd[3]=V[R(i)];
86                 else V[i]->chd[3]=nullNd;
87             }
88             head[u]=merge(head[u], V.front());
89         }
90     }
91     vector<ll> ans;
92     void first_K() {
93         ans.clear(); priority_queue<node> Q;
94         if (dst[s] == -1) return;
95         ans.push_back(dst[s]);
96         if (head[s] != nullNd)
97             Q.push(node(head[s], dst[s]+head[s]->edge->d));
98         for (int _=1; _<k and not Q.empty(); _++) {
99             node p=Q.top(); Q.pop(); ans.push_back(p.d);
100             if (head[p.H->edge->v] != nullNd) {
101                 q.H=head[p.H->edge->v]; q.d=p.d+q.H->edge->d;
102                 Q.push(q);
103             }
104             for (int i=0; i<4; i++)
105                 if (p.H->chd[i] != nullNd) {

```

```

106     q.H=p.H->chd[i];
107     q.d=p.d-p.H->edge->d+p.H->chd[i]->edge->d;
108     Q.push(q);
109 } } }
110 void solve() { // ans[i] stores the i-th shortest path
111     dijkstra(); build();
112     first_K(); // ans.size() might less than k
113 }
114 } solver;

```

6.9 SCC - Tarjan

```

1 // Author: Ian
2 // tarjan SCC
3 void solve() {
4     V<bool> ins(n, false);
5     V<int> scc(n), dfn(n, -1), low(n, inf);
6     stack<int> s;
7     function<void(int)> dfs = [&](int x) {
8         if (~dfn[x]) return;
9         static int t = 0;
10        dfn[x] = low[x] = t++;
11        s.push(x), ins[x] = true;
12        for (auto i : e[x])
13            if (dfs(i), ins[i])
14                low[x] = min(low[x], low[i]);
15        if (dfn[x] == low[x]) {
16            static int ncnt = 0;
17            int p; do {
18                ins[p = s.top()] = false;
19                s.pop(), scc[p] = ncnt;
20            } while (p != x); ncnt++;
21        }
22    };
23    for (int i = 0; i < n; i++)
24        dfs(i);
25 }

```

6.10 2SAT

```

1 // Author: CRYPTOGRAPHER
2 struct TwoSAT {
3     int n;
4     Scc scc;
5     void init(int _n) {
6         // (0,1), (2,3), ...
7         n = _n; scc.init(n*2);
8     }
9     void add_disjunction(int a, int na, int b, int nb) {
10        a = 2*a^na, b = 2*b^nb;
11        scc.addEdge(a^1, b);
12        scc.addEdge(b^1, a);
13    }
14    vector<int> solve() {
15        scc.solve();
16        vector<int> assignment(n, 0);
17        for (int i=0; i<n; i++) {
18            if (scc.bln[2*i] == scc.bln[2*i+1]) return {};
19            assignment[i] = scc.bln[2*i] > scc.bln[2*i+1];
20        }
21        return assignment;
22    }
23 };

```

7 Tree

7.1 Tree Isomorphism (Rooted Trees)

```

1 // Author: Gino
2 // Checks if 2 rooted trees are isomorphic
3 // Time: O(V log V)
4 // vector<vector<int>> G1, G2;
5 // int root_1, root_2;
6 map<vector<int>, int> dict;
7 int encode(const auto& G, int u, int pa = -1) {
8     vector<int> codes;
9     for (auto& v : G[u])
10        if (v != pa)
11            codes.emplace_back(encode(G, v, u));
12    ranges::sort(codes);
13    auto [it, _] = dict.try_emplace(codes, dict.size());
14    return it->second;
15 }
16 bool isomorphic() {
17     dict.clear(); dict[{}] = 0;
18     return encode(G1, root_1) == encode(G2, root_2);
19 }

```

7.2 Tree Isomorphism (Unrooted Trees)

Find the centroid(s) of T_1, T_2 .

Case 1: T_1, T_2 have different number of centroids $\rightarrow$ NO

Case 2: T_1 has centroid c_1, T_2 has centroid c_2

$\rightarrow$ rooted_isomorphic(c_1, c_2)

Case 3: T_1 has centroids c_1, c'_1, T_2 has centroids c_2, c'_2

$\rightarrow$ rooted_isomorphic(c_1, c_2) || rooted_isomorphic(c'_1, c'_2)

7.3 Heavy Light Decomposition

```

1 // Author: Ian
2 void build(V<int>&v);
3 void modify(int p, int k);
4 int query(int ql, int qr);
5 // Insert [ql, qr) segment tree here
6 void solve() {
7     // Input with adjacency list ...
8     V<int> d(n, 0), f(n, 0), sz(n, 1), son(n, -1);
9     F<void(int, int)> dfs1 = [&](int x, int pre) {
10        for (auto i : e[x]) if (i != pre) {
11            d[i] = d[x]+1, f[i] = x;
12            dfs1(i, x), sz[x] += sz[i];
13            if (son[x] == -1 || sz[son[x]] < sz[i])
14                son[x] = i;
15        }
16    }; dfs1(0, 0);
17     V<int> top(n, 0), dfn(n, -1);
18     F<void(int, int)> dfs2 = [&](int x, int t) {
19        static int cnt = 0;
20        dfn[x] = cnt++, top[x] = t;
21        if (son[x] == -1) return;
22        dfs2(son[x], t);
23        for (auto i : e[x]) if (!dfn[i])
24            dfs2(i, i);
25    }; dfs2(0, 0);
26     V<int> dfnv(n);
27     for (int i = 0; i < n; i++)
28         dfnv[dfn[i]] = v[i];
29     build(dfnv);
30     while (q--) {
31         int op, a, b, ans; cin >> op >> a >> b;
32         switch (op) {
33             case 1:
34                 modify(dfn[a-1], b);
35                 break;
36             case 2:
37                 a--, b--, ans = 0;
38                 while (top[a] != top[b]) {
39                     if (d[top[a]] > d[top[b]]) swap(a, b);
40                     ans = max(ans, query(dfn[top[b]], dfn[b]+1));
41                     b = f[top[b]];
42                 }
43                 if (dfn[a] > dfn[b]) swap(a, b);
44                 ans = max(ans, query(dfn[a], dfn[b]+1));
45                 cout << ans << endl;
46                 break;
47             }
48     }
49 }

```

7.4 Virtual Tree

```

1 // Author: std_abs
2 // need lca, in, out
3 vector<pii> virtual_tree(vector<int>&v) {
4     auto cmp = [&](int x, int y) { return in[x] < in[y]; };
5     sort(all(v), cmp);
6     for (int i = 1; i < sz(v); ++i)
7         v.pb(lca(v[i-1], v[i]));
8     sort(all(v), cmp);
9     v.resize(unique(all(v)) - v.begin());
10    vector<int> stk(1, v[0]);
11    vector<pii> res;
12    for (int i = 1; i < sz(v); ++i) {
13        int x = v[i];
14        while (out[stk.back()] < out[x]) stk.pop_back();
15        res.emplace_back(stk.back(), x), stk.pb(x);
16    }
17    return res;
18 }

```

8 Matching

8.1 Bipartite Matching

```

1 // Author: Gino
2 // Function: Max Bipartite Matching in O(V sqrt(E))
3 // Usage:
4 // >>> init(nx, ny) -> add(x, y (+nx))
5 // >>> hk.max_matching() := the matching plan stores in mx,
6 //                               my
7 // >>> hk.min_vertex_cover() := the vertex cover plan stores
8 //                               in vcover
9 // (!) vertices are 0-based: X = [0, nx), Y = [nx, nx+ny)
10 #define pb emplace_back
11 struct HopcroftKarp {
12     int n, nx, ny;

```

```

11 vector<vector<int>> > G;
12 vector<int> mx, my;
13 void init(int _nx, int _ny) {
14     nx = _nx, ny = _ny;
15     n = nx + ny;
16     G.assign(n, vector<int>());
17 }
18 void add(int x, int y) { G[x].pb(y); G[y].pb(x); }
19 int max_matching() {
20     vector<int> dis, vis;
21     mx.assign(n, -1); my.assign(n, -1);
22     function<bool(int)> dfs = [&](int x) {
23         vis[x] = 1;
24         for (int y : G[x]) {
25             int p = my[y];
26             if (p == -1 || (dis[p] == dis[x] + 1 && !vis[p] &&
27                 dfs(p)))
28                 return mx[x] = y, my[y] = x, true;
29         }
30         return false;
31     };
32     while (true) {
33         dis.assign(n, -1);
34         queue<int> q;
35         for (int x = 0; x < nx; x++)
36             if (mx[x] == -1) dis[x] = 0, q.push(x);
37         while (!q.empty()) {
38             int x = q.front(); q.pop();
39             for (int y : G[x])
40                 if (my[y] != -1 && dis[my[y]] == -1)
41                     dis[my[y]] = dis[x] + 1, q.push(my[y]);
42         }
43         vis.assign(n, 0);
44         bool found = false;
45         for (int x = 0; x < nx; x++)
46             if (mx[x] == -1 && dfs(x)) found = true;
47         if (!found) break;
48     }
49     int ans = 0;
50     for (int x = 0; x < nx; x++) if (mx[x] != -1) ans++;
51     return ans;
52 }
53 vector<int> vcover;
54 int min_vertex_cover() {
55     int ans = max_matching();
56     vector<int> vis(n, 0);
57     function<void(int)> dfs = [&](int x) {
58         vis[x] = true;
59         for (int y : G[x])
60             if (y == mx[x] || my[y] == -1 || vis[y]) continue;
61         vis[y] = true;
62         dfs(my[y]);
63     };
64     for (int x = 0; x < nx; x++) if (mx[x] == -1) dfs(x);
65     vcover.clear();
66     for (int x = 0; x < nx; x++) if (!vis[x]) vcover.pb(x);
67     for (int y = nx; y < nx + ny; y++) if (vis[y])
68         vcover.pb(y);
69     return ans;
70 }
71 hk;

```

8.2 Bipartite Weighted Matching

```

1 // Author: ckiseki
2 // n = #(left vertices) = #(right vertices)
3 // left, right vertices labeled 1 ~ n separately
4 // 1. build 1-based matrix G of size (n + 1) * (n + 1)
5 // G[u][v] := weight between (left u, right v)
6 // <!-- if not need perfect matching => set weights of all
7 // negative edges to 0
8 // 2. constrcut (solve at same time): KM km(n, G);
9 // => km.ans (max perfect weight sum)
10 // => km.fl[u] (left u -> right v)
11 // => km.fr[v] (right v -> left u)
12 // Time: O(V^3), Space: O(V^2)
13 // Test: V <= 500, 243ms
14 struct KM { // maximize, test @ UOJ 80
15     int n, l, r; ll ans; // fl and fr are the match
16     vector<ll> hl, hr; vector<int> fl, fr, pre, q;
17     void bfs(const auto &w, int s) {
18         vector<int> vl(n + 1), vr(n + 1); vector<ll> slk(n + 1,
19             LINF);
20         l = r = 0; vr[q[r++] = s] = true;
21         auto check = [&](int x) -> bool {
22             if (vl[x] || slk[x] > 0) return true;
23             vl[x] = true; slk[x] = LINF;
24             if (fl[x] != -1) return (vr[q[r++] = fl[x]] = true);
25             while (x != -1) swap(x, fr[fl[x] = pre[x]]);

```

```

24         return false;
25     };
26     while (true) {
27         while (l < r)
28             for (int x = 1, y = q[l++]; x <= n; ++x) if (!vl[x])
29                 if (ll val = hl[x] + hr[y] - w[x][y]; val <
30                     slk[x]) {
31                     slk[x] = val;
32                     if (pre[x] = y, !check(x)) return;
33                 }
34         ll d = LINF;
35         for (int x = 1; x <= n; ++x) d = min(d, slk[x]);
36         for (int x = 1; x <= n; ++x)
37             vl[x] ? hl[x] += d : slk[x] -= d;
38         for (int x = 1; x <= n; ++x) if (vr[x]) hr[x] -= d;
39         for (int x = 1; x <= n; ++x) if (!check(x)) return;
40     }
41     KM(int n_, const auto &w) : n(n_), ans(0),
42         hl(n + 1), hr(n + 1), fl(n + 1, -1), fr(fl), pre(n + 1),
43         q(n + 1) {
44         for (int i = 1; i <= n; ++i) {
45             hl[i] = -LINF;
46             for (int j = 1; j <= n; ++j) hl[i] = max(hl[i],
47                 (ll)w[i][j]);
48         }
49         for (int i = 1; i <= n; ++i) bfs(w, i);
50         for (int i = 1; i <= n; ++i) ans += w[i][fl[i]];
51     };

```

8.3 General Matching

```

1 // Author: ckiseki
2 // 0. build 1-based graph vector<vector<int>> G(n + 1);
3 // 1. construct (solve at same time):
4 //     GeneralMatching M(G, n); => M.ans
5 // status: M.match[x] (== -1 if not matched, 1 <= x <= n)
6 // Time: O(V^3), Space: O(V + E)
7 // Test: V <= 500, 11ms
8 struct GeneralMatching {
9     queue<int> q; int ans, n;
10    vector<int> fa, s, v, pre, match;
11    int Find(int u) {
12        return u == fa[u] ? u : fa[u] = Find(fa[u]);
13    }
14    int LCA(int x, int y) {
15        static int tk = 0; tk++; x = Find(x); y = Find(y);
16        for (; swap(x, y)) if (x != 0) {
17            if (v[x] == tk) return x;
18            v[x] = tk;
19            x = Find(pre[match[x]]);
20        }
21    }
22    void Blossom(int x, int y, int l) {
23        for (; Find(x) != l; x = pre[y]) {
24            pre[x] = y, y = match[x];
25            if (s[y] == 1) q.push(y), s[y] = 0;
26            for (int z : {x, y}) if (fa[z] == z) fa[z] = l;
27        }
28    }
29    bool Bfs(auto &g, int r) {
30        iota(all(fa), 0); ranges::fill(s, -1);
31        q = queue<int>(); q.push(r); s[r] = 0;
32        for (; !q.empty(); q.pop()) {
33            for (int x = q.front(); int u : g[x])
34                if (s[u] == -1) {
35                    if (pre[u] = x, s[u] = 1, match[u] == 0) {
36                        for (int a = u, b = x, last;
37                            b != 0; a = last, b = pre[a])
38                            last = match[b], match[b] = a, match[a] = b;
39                        return true;
40                    }
41                    q.push(match[u]); s[match[u]] = 0;
42                } else if (!s[u] && Find(u) != Find(x)) {
43                    int l = LCA(u, x);
44                    Blossom(x, u, l); Blossom(u, x, l);
45                }
46            }
47        return false;
48    }
49    GeneralMatching(auto &g, int n_) : ans(0), n(n_),
50        fa(n + 1), s(n + 1), v(n + 1), pre(n + 1, 0), match(n + 1,
51            0) {
52        for (int x = 1; x <= n; ++x)
53            if (match[x] == 0) ans += Bfs(g, x);
54        for (int x = 1; x <= n; ++x)
55            if (match[x] == 0) match[x] = -1;
56    }; // tested @ yosupo judge

```


8.4 General Weighted Matching

```

1 // Author: ckiseki
2 // 1. GeneralWeightedMatching M(n);
3 // (1-based, 1 <= u, v <= n)
4 // 2. add edges: M.set_edge(u, v, w);
5 // 3. auto [tot_weight, n_matches] = M.solve();
6 // 4. M.match[u] (== -1 if not matched, 1-based index if
   matched)
7 // Time: O(V^3), Space: O(V^2)
8 // Test: V <= 500, 369ms
9 struct GeneralWeightedMatching { // 1-based
10     static const int inf = INT_MAX;
11     struct edge { int u, v, w; }; int n, nx;
12     vector<int> lab; vector<vector<edge>> g;
13     vector<int> slack, match, st, pa, S, vis;
14     vector<vector<int>> flo, flo_from; queue<int> q;
15     GeneralWeightedMatching(int n_) : n(n_), nx(n * 2), lab(nx
   + 1),
16         g(nx + 1, vector<edge>(nx + 1)), slack(nx + 1),
17         flo(nx + 1, flo_from(nx + 1, vector(n + 1, 0))) {
18         match = st = pa = S = vis = slack;
19         REP(u, 1, n) REP(v, 1, n) g[u][v] = {u, v, 0};
20     }
21     int ED(edge e) {
22         return lab[e.u] + lab[e.v] - g[e.u][e.v].w * 2; }
23     void update_slack(int u, int x, int &s) {
24         if (!s || ED(g[u][x]) < ED(g[s][x])) s = u; }
25     void set_slack(int x) {
26         slack[x] = 0;
27         REP(u, 1, n)
28             if (g[u][x].w > 0 && st[u] != x && S[st[u]] == 0)
29                 update_slack(u, x, slack[x]);
30     }
31     void q_push(int x) {
32         if (x <= n) q.push(x);
33         else for (int y : flo[x]) q_push(y);
34     }
35     void set_st(int x, int b) {
36         st[x] = b;
37         if (x > n) for (int y : flo[x]) set_st(y, b);
38     }
39     vector<int> split_flo(auto &f, int xr) {
40         auto it = find(all(f), xr);
41         if (auto pr = it - f.begin(); pr % 2 == 1)
42             reverse(1 + all(f), it = f.end() - pr);
43         auto res = vector(f.begin(), it);
44         return f.erase(f.begin(), it), res;
45     }
46     void set_match(int u, int v) {
47         match[u] = g[u][v].v;
48         if (u <= n) return;
49         int xr = flo_from[u][g[u][v].u];
50         auto &f = flo[u], z = split_flo(f, xr);
51         REP(i, 0, int(z.size())-1) set_match(z[i], z[i ^ 1]);
52         set_match(xr, v); f.insert(f.end(), all(z));
53     }
54     void augment(int u, int v) {
55         for (;;) {
56             int xnv = st[match[u]]; set_match(u, v);
57             if (!xnv) return;
58             set_match(v = xnv, u = st[pa[xnv]]);
59         }
60     }
61     /* SPLIT_HASH_HERE */
62     int lca(int u, int v) {
63         static int t = 0; ++t;
64         for (++t; u || v; swap(u, v)) if (u) {
65             if (vis[u] == t) return u;
66             vis[u] = t; u = st[match[u]];
67             if (u) u = st[pa[u]];
68         }
69         return 0;
70     }
71     void add_blossom(int u, int o, int v) {
72         int b = int(find(n + 1 + all(st), 0) - begin(st));
73         lab[b] = 0, S[b] = 0; match[b] = match[o];
74         vector<int> f = {o};
75         for (int x : {u, v}) {
76             for (int y; x != o; x = st[pa[y]])
77                 f.pb(x), f.pb(y = st[match[x]]), q_push(y);
78             reverse(1 + all(f));
79         }
80         flo[b] = f; set_st(b, b);
81         REP(x, 1, nx) g[b][x].w = g[x][b].w = 0;
82         REP(x, 1, n) flo_from[b][x] = 0;
83         for (int xs : flo[b]) {
84             REP(x, 1, nx)
85                 if (g[b][x].w == 0 || ED(g[xs][x]) < ED(g[b][x]))
86                     g[b][x] = g[xs][x], g[x][b] = g[x][xs];
87             REP(x, 1, n)
88                 if (flo_from[xs][x]) flo_from[b][x] = xs;
89         }
90         set_slack(b);
91     }
92     void expand_blossom(int b) {
93         for (int x : flo[b]) set_st(x, x);
94         int xr = flo_from[b][g[b][pa[b]].u], xs = -1;
95         for (int x : split_flo(flo[b], xr)) {
96             if (xs == -1) { xs = x; continue; }
97             pa[xs] = g[x][xs].u; S[xs] = 1, S[x] = 0;
98             slack[xs] = 0; set_slack(x); q_push(x); xs = -1;
99         }
100         for (int x : flo[b])
101             if (x == xr) S[x] = 1, pa[x] = pa[b];
102             else S[x] = -1, set_slack(x);
103         st[b] = 0;
104     }
105     bool on_found_edge(const edge &e) {
106         if (int u = st[e.u], v = st[e.v]; S[v] == -1) {
107             int nu = st[match[v]]; pa[v] = e.u; S[v] = 1;
108             slack[v] = slack[nu] = 0; S[nu] = 0; q_push(nu);
109         } else if (S[v] == 0) {
110             if (int o = lca(u, v)) add_blossom(u, o, v);
111             else return augment(u, v), augment(v, u), true;
112         }
113         return false;
114     }
115     /* SPLIT_HASH_HERE */
116     bool matching() {
117         ranges::fill(S, -1); ranges::fill(slack, 0);
118         q = queue<int>();
119         REP(x, 1, nx) if (st[x] == x && !match[x])
120             pa[x] = 0, S[x] = 0, q_push(x);
121         if (q.empty()) return false;
122         for (;;) {
123             while (q.size()) {
124                 int u = q.front(); q.pop();
125                 if (S[st[u]] == 1) continue;
126                 REP(v, 1, n)
127                     if (g[u][v].w > 0 && st[u] != st[v]) {
128                         if (ED(g[u][v]) != 0)
129                             update_slack(u, st[v], slack[st[v]]);
130                         else if (on_found_edge(g[u][v])) return true;
131                     }
132             }
133             int d = inf;
134             REP(b, n + 1, nx) if (st[b] == b && S[b] == 1)
135                 d = min(d, lab[b] / 2);
136             REP(x, 1, nx)
137                 if (int s = slack[x]; st[x] == x && s && S[x] <= 0)
138                     d = min(d, ED(g[s][x]) / (S[x] + 2));
139             REP(u, 1, n)
140                 if (S[st[u]] == 1) lab[u] += d;
141                 else if (S[st[u]] == 0) {
142                     if (lab[u] <= d) return false;
143                     lab[u] -= d;
144                 }
145             REP(b, n + 1, nx) if (st[b] == b && S[b] >= 0)
146                 lab[b] += d * (2 - 4 * S[b]);
147             REP(x, 1, nx)
148                 if (int s = slack[x]; st[x] == x &&
149                     s && st[s] != x && ED(g[s][x]) == 0)
150                     if (on_found_edge(g[s][x])) return true;
151             REP(b, n + 1, nx)
152                 if (st[b] == b && S[b] == 1 && lab[b] == 0)
153                     expand_blossom(b);
154         }
155         return false;
156     }
157     pair<ll, int> solve() {
158         ranges::fill(match, 0);
159         REP(u, 0, n) st[u] = u, flo[u].clear();
160         int w_max = 0;
161         REP(u, 1, n) REP(v, 1, n) {
162             flo_from[u][v] = (u == v ? u : 0);
163             w_max = max(w_max, g[u][v].w);
164         }
165         REP(u, 1, n) lab[u] = w_max;
166         int n_matches = 0; ll tot_weight = 0;
167         while (matching()) ++n_matches;
168         REP(u, 1, n) if (match[u] && match[u] < u)
169             tot_weight += g[u][match[u]].w;
170         REP(u, 1, n) if (match[u] == 0) match[u] = -1;
171         return make_pair(tot_weight, n_matches);
172     }
173     void set_edge(int u, int v, int w) {

```

```
174 g[u][v].w = g[v][u].w = w; }
175 };
```

9 Flow

9.1 Flow Methods

```
1 // Author: CRyptographER (some modified by Gino)
2 Maximize  $c^T x$  subject to  $Ax \leq b, x \geq 0$ ;
3 with the corresponding symmetric dual problem,
4 Minimize  $b^T y$  subject to  $A^T y \geq c, y \geq 0$ .
5
6 Maximize  $c^T x$  subject to  $Ax \leq b$ ;
7 with the corresponding asymmetric dual problem,
8 Minimize  $b^T y$  subject to  $A^T y = c, y \geq 0$ .
9
10 Maximize  $\sum x$  subject to  $x_i + x_j \leq A_{ij}, x \geq 0$ ;
11 => Maximize  $\sum x$  subject to  $x_i + x_j \leq A_{ij}$ ;
12 => Minimize  $A^T y = \sum A_{ij} y_{ij}$  subject to for all  $v$ ,
     $\sum_{i=v \text{ or } j=v} y_{ij} = 1, y_{ij} \geq 0$ 
13 => possible optimal solution:  $y_{ij} = \{0, 0.5, 1\}$ 
14 =>  $y' = 2y$ :  $\sum_{i=v \text{ or } j=v} y'_{ij} = 2, y'_{ij} = \{0, 1, 2\}$ 
15 => Minimum Bipartite perfect matching/2 ( $V1=X, V2=X, E=A$ )
16
17 General Graph:
18 |Max Ind. Set| + |Min Vertex Cover| = |V|
19 |Max Ind. Edge Set| + |Min Edge Cover| = |V|
20 Bipartite Graph:
21 |Max Ind. Set| = |Min Edge Cover|
22 |Max Ind. Edge Set| = |Min Vertex Cover|
23
24 To reconstruct the minimum vertex cover, dfs from each
25 unmatched vertex on the left side and with unused edges
26 only. Equivalently, dfs from source with unused edges
27 only and without visiting sink. Then, a vertex is
28 chosen iff. it is on the left side and without visited
29 or on the right side and visited through dfs.
30
31 Minimum Weighted Bipartite Edge Cover:
32 Construct new bipartite graph with  $n+m$  vertices on each
    side:
33 for each vertex  $u$ , duplicate a vertex  $u'$  on the other side
34 for each edge  $(u,v,w)$ , add edges  $(u,v,w)$  and  $(v',u',w)$ 
35 for each vertex  $u$ , add edge  $(u,u',2w)$  where  $w$  is min edge
    connects to  $u$ 
36 then the answer is the minimum perfect matching of the new
    graph (KM)
37
38 Maximum density subgraph (  $\sum W_e + \sum W_v$  ) / |V|
39 Binary search on answer:
40 For a fixed  $D$ , construct a Max flow model as follow:
41 Let  $S$  be Sum of all weight( or inf)
42 1. from source to each node with cap =  $S$ 
43 2. For each  $(u,v,w)$  in  $E$ ,  $(u \rightarrow v, \text{cap}=w), (v \rightarrow u, \text{cap}=w)$ 
44 3. For each node  $v$ , from  $v$  to sink with cap =  $S + 2 * D - \text{deg}[v] - 2 * (W \text{ of } v)$ 
45 where  $\text{deg}[v] = \sum \text{weight of edge associated with } v$ 
46 If  $\text{maxflow} < S * |V|$ ,  $D$  is an answer.
47
48 Requiring subgraph: all vertex can be reached from source
    with
49 edge whose cap  $> 0$ .
50
51 Maximum closed subgraph
52 1. connect source with positive weighted
    vertex(capacity=weight)
53 2. connect sink with negative weighted vertex(capacity=-
    weight)
54 3. make capacity of the original edges = inf
55 4. ans = sum(positive weighted vertex weight) - (max flow)
56
57 (Node-disjoint) Min DAG Path Cover (用最少路徑覆蓋所有點)
58 Node disjoint: 拆出來的路徑不能共用同一個點
59 將一個點  $u$  裂成  $u+$  和  $u-$ , 代表進入和出去
60 對於一條邊  $(u, v)$  建邊  $u- \Rightarrow v+$ 
61 1.  $+$ 點們和  $-$ 點們形成一張二分圖
62 2. 最差的答案是  $n$ , 代表每個點自己一個點就是一條路徑
63 3. 只要一組  $(u-, v+)$  匹配成功那對應到的答案恰好會  $-1$ 
64 >>> ans =  $n -$  這張二分圖的最大匹配
65
66 General DAG Path Cover
67 跟 Node-disjoint 版本差在點可以被很多條路共用
68 建邊方式 Node-disjoint 差別在條件比較鬆
69 Node-disjoint:  $u- \Rightarrow v+$  只能在  $(u, v)$  有邊時建
70 General:  $u- \Rightarrow v+$  在  $u$  能走到  $v$  的時候就建
71
72 Dilworth Theorem
73 反鏈：一些節點的集合，滿足這些節點互相無法抵達
74 最大反鏈 = 最小 General DAG Path Cover
```

9.2 Dinic

```
1 // Author: Benson (Extensions by Gino)
2 // Function: Max Flow,  $O(V^2 E)$ 
```

```
3 // Usage: Call init(n) first, then add(u, v, w) based on
    your model
4 // (!) vertices 0-based
5 // >>> flow() := return max flow
6 // >>> find_cut() := return min cut + store cut set in
    dinic.cut
7 // >>> find_matching() := return |M| + store matching plan
    in dinic.matching
8 // >>> flow_decomposition := return max flow + store
    decomposition in dinic.D
9 #define int long long
10 #define eb emplace_back
11 #define ALL(a) a.begin(), a.end()
12 struct Dinic {
13     struct Edge {
14         // t: to | C: original capacity | c: current capacity |
        r: residual edge | f: current flow
15         int t, C, c, r, f;
16         bool fw; // is in forward-edge graph
17         Edge() {}
18         Edge(int t, int C, int r, bool fw, int f=0):
            t(t), C(C), c(C), r(r), fw(fw), f(f) {}
19     };
20     vector<vector<Edge>> G;
21     vector<int> dis, iter;
22     int n, s, t;
23     void init(int _n) {
24         n = _n;
25         G.resize(n), dis.resize(n), iter.resize(n);
26         for(int i = 0; i < n; ++i)
27             G[i].clear();
28     }
29     void add(int a, int b, int c) {
30         G[a].eb(b, c, G[b].size(), true);
31         G[b].eb(a, 0, G[a].size() - 1, false);
32     }
33     bool bfs() {
34         fill(ALL(dis), -1);
35         dis[s] = 0;
36         queue<int> que;
37         que.push(s);
38         while(!que.empty()) {
39             int u = que.front(); que.pop();
40             for(auto& e : G[u]) {
41                 if(e.c > 0 && dis[e.t] == -1) {
42                     dis[e.t] = dis[u] + 1;
43                     que.push(e.t);
44                 }
45             }
46         }
47         return dis[t] != -1;
48     }
49     int dfs(int u, int cur) {
50         if(u == t) return cur;
51         for(int &i = iter[u]; i < (int)G[u].size(); ++i) {
52             auto& e = G[u][i];
53             if(e.c > 0 && dis[u] + 1 == dis[e.t]) {
54                 int ans = dfs(e.t, min(cur, e.c));
55                 if(ans > 0) {
56                     G[e.t][e.r].c += ans;
57                     e.c -= ans;
58                     return ans;
59                 }
60             }
61         }
62         return 0;
63     }
64     // find max flow
65     int flow(int a, int b) {
66         s = a, t = b;
67         int ans = 0;
68         while(bfs()) {
69             fill(ALL(iter), 0);
70             int tmp;
71             while((tmp = dfs(s, INF)) > 0)
72                 ans += tmp;
73         }
74         return ans;
75     }
76     // min cut plan
77     vector<pair<pair<int, int>, int>> cut;
78     int find_cut(int a, int b) {
79         int cut_sz = flow(a, b);
80         vector<int> vis(n, 0);
81         cut.clear();
82         function<void(int)> dfs = [&](int u) {
83             vis[u] = 1;
84             for(auto& e : G[u])
85                 if(e.c > 0 && !vis[e.t])
86                     dfs(e.t);
87         };
88         dfs(a);
89         for(int u = 0; u < n; u++)
```

```

87     if (vis[u])
88         for (auto& e : G[u])
89             if (!vis[e.t])
90                 cut.eb(make_pair(make_pair(u, e.t), G[e.t]
[e.r].c));
91     return cut.sz;
92 }
93 // bipartite matching plan
94 vector<pair<int, int>> matching;
95 int find_matching(int Xstart, int Xend, int Ystart, int
Yend, int a, int b) {
96     int msz = flow(a, b);
97     matching.clear();
98     for (int x = Xstart; x <= Xend; x++)
99         for (auto& e : G[x])
100             if (e.c == 0 && Ystart <= e.t && e.t <= Yend)
101                 matching.emplace_back(make_pair(x, e.t));
102     return msz;
103 }
104 // flow decomposition
105 vector<pair<int, vector<int>>> D; // (flow amount, [path
pl ... pk])
106 int flow_decomposition(int a, int b) {
107     int mxflow = flow(a, b);
108
109     vector<vector<Edge>> fG(n); // graph consists of forward
edges
110     for (int u = 0; u < n; u++) {
111         for (auto& e : G[u]) {
112             if (e.fw) {
113                 e.f = e.C - e.c;
114                 if (e.f > 0) fG[u].eb(e);
115             }
116         }
117     }
118     vector<int> vis;
119     function<int(int, int)> dfs = [&](int u, int cur) {
120         if (u == b) {
121             D.back().second.eb(u);
122             return cur;
123         }
124         vis[u] = 1;
125         for (auto& e : fG[u]) {
126             if (e.f > 0 && !vis[e.t]) {
127                 int ans = dfs(e.t, min(cur, e.f));
128                 if (ans > 0) {
129                     e.f -= ans;
130                     D.back().second.eb(u);
131                     return ans;
132                 }
133             }
134         }
135         D.clear();
136         int quota = mxflow;
137         while (quota > 0) {
138             D.emplace_back(make_pair(0, vector<int>()));
139             vis.assign(n, 0);
140             int f = dfs(a, INF);
141             if (f == 0) break;
142             reverse(D.back().second.begin(),
D.back().second.end());
143             D.back().first = f, quota -= f;
144         }
145         return mxflow;
146     };

```

9.3 ISAP

```

1 // Author: CRYPTOGRAPHER
2 // 1-based: vertices are numbered 1 ~ n
3 // 1. flow.init(n, s, t);
4 // n = #vertices, s, t = source, sink
5 // 2. flow.addEdge(u, v, c); // u, v in [1, n]
6 // 3. call: int ans = flow.flow();
7 // => ans = max flow value from s to t
8 // Time: O(V^2 * E) worst case, usually much faster in
practice
9 static const int MAXV=50010;
10 static const int INF =1000000;
11 struct Maxflow{
12     struct Edge{
13         int v, c, r;
14         Edge(int _v, int _c, int _r):v(_v),c(_c),r(_r){}
15     };
16     int s, t; vector<Edge> G[MAXV];
17     int iter[MAXV], d[MAXV], gap[MAXV], tot;
18     void init(int n, int s, int t){
19         tot=n, s=s, t=t;
20         for(int i=1; i<=tot; i++){
21             G[i].clear(); iter[i]=d[i]=gap[i]=0;

```

```

22     }
23 }
24 void addEdge(int u, int v, int c){
25     G[u].push_back(Edge(v, c, SZ[G[v]]));
26     G[v].push_back(Edge(u, 0, SZ[G[u]]-1));
27 }
28 int DFS(int p, int flow){
29     if(p==t) return flow;
30     for(int &i=iter[p]; i<SZ[G[p]]; i++){
31         Edge &e=G[p][i];
32         if(e.c>0&&d[p]==d[e.v]+1){
33             int f=DFS(e.v, min(flow, e.c));
34             if(f){ e.c-=f; G[e.v][e.r].c+=f; return f; }
35         }
36     }
37     if(--gap[d[p]]==0) d[s]=tot;
38     else{ d[p]++; iter[p]=0; ++gap[d[p]]; }
39     return 0;
40 }
41 int flow(){
42     int res=0;
43     for(res=0, gap[0]=tot; d[s]<tot; res+=DFS(s, INF));
44     return res;
45 }
46 void reset(){
47     for(int i=1; i<=tot; i++) iter[i]=d[i]=gap[i]=0;
48 }
49 } flow;

```

9.4 Bounded Max Flow

```

1 // Author: CRYPTOGRAPHER
2 // Max flow with lower/upper bound on edges (source-sink
version)
3 // <!-- 1-based: original vertices are numbered 1 ~ n.
4 // Super source, sink: n+1, n+2 respectively.
5 // <!-- dependency: ISAP.cpp
6 // 1. n = #vertices (1 ~ n), m = #edges
7 // s, t = original source, sink (both in [1, n])
8 // 2. l[i], r[i] := edge i goes from l[i] -> r[i] (l[i],
r[i] in [1, n])
9 // 3. a[i], b[i] := flow on edge i must lie in [a[i], b[i]]
10 // 4. int ans = solve(n, m, s, t);
11 // => ans == -1 : no feasible flow exists
12 // => ans : feasible max flow value
13 // <!-- N, M must be large enough for original graph + 2
super nodes
14 // Time: O(V^2 E), Space: O(V + E)
15 int in[N], out[N], l[M], r[M], a[M], b[M];
16 int solve(int n, int m, int s, int t){
17     flow.init(n+2, n+1, n+2); // super source = n+1, super
sink = n+2
18     for(int i = 0; i < m; i++){
19         in[r[i]] += a[i]; out[l[i]] += a[i];
20         flow.addEdge(l[i], r[i], b[i] - a[i]);
21         // flow from l[i] to r[i] must lie in [a[i], b[i]]
22     }
23     int nd = 0;
24     for(int i = 1; i <= n; i++){
25         if(in[i] < out[i]){
26             flow.addEdge(i, flow.t, out[i] - in[i]);
27             nd += out[i] - in[i];
28         }
29         if(out[i] < in[i])
30             flow.addEdge(flow.s, i, in[i] - out[i]);
31     }
32     flow.addEdge(t, s, INF); // original sink -> source
33     if(flow.flow() != nd) return -1; // infeasible
34     int ans = flow.G[s].back().c; // t->s edge's current flow
= feasible flow s->t
35     flow.G[s].back().c = flow.G[t].back().c = 0;
36     for(size_t i = 0; i < flow.G[flow.s].size(); i++){
37         Maxflow::Edge &e = flow.G[flow.s][i];
38         flow.G[flow.s][i].c = 0; flow.G[e.v][e.r].c = 0;
39     }
40     for(size_t i = 0; i < flow.G[flow.t].size(); i++){
41         Maxflow::Edge &e = flow.G[flow.t][i];
42         flow.G[flow.t][i].c = 0; flow.G[e.v][e.r].c = 0;
43     }
44     flow.addEdge(flow.s, s, INF); flow.addEdge(t, flow.t,
INF);
45     flow.reset(); // keeps G[] intact, only resets search
state
46     return ans + flow.flow();
47 }

```

9.5 MCMF

```

1 // Author: CRYPTOGRAPHER
2 // MCMF (supports negative edges without negative cycles)
3 // <!-- 1-based: vertices are numbered 1 ~ n.
4 // 1. MCMF.init(n); // n = #vertices (1 ~ n)

```

```

5 // 2. MCMF.add(u, v, cap, cost);
6 // 3. auto [max_flow, min_cost] = MCMF.flow(s, t); // s, t
  in [1, n]
7 // Time: O(F * E), Space: O(V + E)
8 typedef int Tcost;
9 const int MAXV = 20010;
10 const int INFf = 1000000;
11 const Tcost INFc = 1e9;
12 struct MCMF {
13     struct Edge {
14         int v, cap;
15         Tcost w;
16         int rev;
17         bool fw;
18         Edge() {}
19         Edge(int t2, int t3, Tcost t4, int t5, bool t6)
20             : v(t2), cap(t3), w(t4), rev(t5), fw(t6) {}
21     };
22     int V, s, t;
23     vector<Edge> G[MAXV];
24     void init(int n) {
25         V = n;
26         for(int i = 0; i <= V; i++) G[i].clear();
27     }
28     void add(int a, int b, int cap, Tcost w) {
29         G[a].push_back(Edge(b, cap, w, (int)G[b].size(), true));
30         G[b].push_back(Edge(a, 0, -w, (int)G[a].size()-1,
31             false));
32     }
33     Tcost d[MAXV];
34     int id[MAXV], mom[MAXV];
35     bool inqu[MAXV];
36     queue<int> q;
37     pair<int, Tcost> flow(int _s, int _t) {
38         s = _s, t = _t;
39         int mxf = 0; Tcost mnc = 0;
40         while(1) {
41             fill(d, d+V, INFc); // need to use type cast
42             fill(inqu, inqu+V, 0);
43             fill(mom, mom+V, -1);
44             mom[s] = s;
45             d[s] = 0;
46             q.push(s); inqu[s] = 1;
47             while(q.size()) {
48                 int u = q.front(); q.pop();
49                 inqu[u] = 0;
50                 for(int i = 0; i < (int) G[u].size(); i++) {
51                     Edge &e = G[u][i];
52                     int v = e.v;
53                     if(e.cap > 0 && d[v] > d[u]+e.w) {
54                         d[v] = d[u]+e.w;
55                         mom[v] = u;
56                         id[v] = i;
57                         if(!inqu[v]) q.push(v), inqu[v] = 1;
58                     }
59                 }
60                 if(mom[t] == -1) break;
61                 int df = INFf;
62                 for(int u = t; u != s; u = mom[u])
63                     df = min(df, G[mom[u]][id[u]].cap);
64                 for(int u = t; u != s; u = mom[u]) {
65                     Edge &e = G[mom[u]][id[u]];
66                     e.cap -= df;
67                     G[e.v][e.rev].cap += df;
68                 }
69                 mxf += df;
70                 mnc += df*d[t];
71             }
72             return make_pair(mxf, mnc);
73         }
74 };

```

9.6 Push-Relabel

```

1 // Author: Simon Lindholm (kactl)
2 // 0-based: vertices are numbered 0 ~ n-1
3 // 1. PushRelabel pr(n); // n = #vertices
4 // 2. pr.addEdge(u, v, c); // u, v in [0, n-1]
5 // 3. ll ans = pr.calc(s, t);
6 // => ans = max flow value from s to t
7 // To obtain actual flow:
8 // for (int u = 0; u < n; u++)
9 //     for (auto& e : pr.g[u])
10 //         if (e.f > 0) => (u, e.dest, e.f)
11 // Time: O(V^2 * sqrt(E)), better on dense graph
12 struct PushRelabel {
13     struct Edge { int dest, back; ll f, c; };
14     vector<vector<Edge>> g;
15     vector<ll> ec;

```

```

16     vector<Edge*> cur;
17     vector<vi> hs; vi H;
18     PushRelabel(int n) : g(n), ec(n), cur(n), hs(2*n), H(n) {}
19
20     void addEdge(int s, int t, ll cap, ll rcap=0) {
21         if (s == t) return;
22         g[s].push_back({t, sz(g[t]), 0, cap});
23         g[t].push_back({s, sz(g[s])-1, 0, rcap});
24     }
25     void addFlow(Edge& e, ll f) {
26         Edge &back = g[e.dest][e.back];
27         if (!ec[e.dest] && f) hs[H[e.dest]].push_back(e.dest);
28         e.f += f; e.c -= f; ec[e.dest] += f;
29         back.f -= f; back.c += f; ec[back.dest] -= f;
30     }
31     ll calc(int s, int t) {
32         int v = sz(g); H[s] = v; ec[t] = 1;
33         vi co(2*v); co[0] = v-1;
34         rep(i, 0, v) cur[i] = g[i].data();
35         for (Edge& e : g[s]) addFlow(e, e.c);
36
37         for (int hi = 0;;) {
38             while (hs[hi].empty()) if (!hi--) return -ec[s];
39             int u = hs[hi].back(); hs[hi].pop_back();
40             while (ec[u] > 0) // discharge u
41                 if (cur[u] == g[u].data() + sz(g[u])) {
42                     H[u] = 1e9;
43                     for (Edge& e : g[u]) if (e.c && H[u] >
44                         H[e.dest]+1)
45                         H[u] = H[e.dest]+1, cur[u] = &e;
46                     if (++co[H[u]], !--co[hi] && hi < v)
47                         rep(i, 0, v) if (hi < H[i] && H[i] < v)
48                             --co[H[i]], H[i] = v + 1;
49                     hi = H[u];
50                 } else if (cur[u]->c && H[u] == H[cur[u]->dest]+1)
51                     addFlow(*cur[u], min(ec[u], cur[u]->c));
52                 else ++cur[u];
53             }
54         bool leftOfMinCut(int a) { return H[a] >= sz(g); }
55 };

```

9.7 Gomory-Hu Tree

```

1 // Author: chilli, Takanori MAEHARA (kactl)
2 // <!> Dependency: PushRelabel.cpp
3 // Given a list of edges representing an undirected flow
  graph,
4 // returns edges of the Gomory-Hu tree.
5 // maxflow / mincut of (u, v) => min edge from u to v on
  Gomory-Hu tree
6 // Time: O(V) * O(PushRelabel)
7 typedef array<ll, 3> Edge; // (u, v, w)
8 vector<Edge> gomoryHu(int N, vector<Edge> ed) {
9     vector<Edge> tree;
10     vi par(N);
11     rep(i, 1, N) {
12         PushRelabel D(N); // Dinic also works
13         for (Edge t : ed) D.addEdge(t[0], t[1], t[2], t[2]);
14         tree.push_back({i, par[i], D.calc(i, par[i])});
15         rep(j, i+1, N)
16             if (par[j] == par[i] && D.leftOfMinCut(j)) par[j] = i;
17     }
18     return tree;
19 }

```

9.8 Global Min Cut

```

1 // Author: ckiseki
2 // 1-based: vertices are numbered 1 ~ n
3 // 1. vector<vector<ll>> G(n+1, vector<ll>(n+1, 0));
4 // 2. add_edge(G, u, v, c); // add undirected edge u-v with
  weight c, u,v in [1,n]
5 // 3. ll ans = mincut(G, n); // n = #vertices
6 // => ans = global min cut value
7 // Time: O(V^3)
8 void add_edge(auto &w, int u, int v, int c) {
9     w[u][v] += c; w[v][u] += c; }
10 auto phase(const auto &w, int n, vector<int> id) {
11     vector<ll> g(n+1); int s = -1, t = -1;
12     while (!id.empty()) {
13         int c = -1;
14         for (int i : id) if (c == -1 || g[i] > g[c]) c = i;
15         s = t; t = c;
16         id.erase(ranges::find(id, c));
17         for (int i : id) g[i] += w[c][i];
18     }
19     return tuple{s, t, g[t]};
20 }
21 ll mincut(auto w, int n) {
22     ll cut = numeric_limits<ll>::max();
23     vector<int> id(n); iota(all(id), 1);

```



```

24 for (int i = 0; i < n - 1; ++i) {
25     auto [s, t, gt] = phase(w, n, id);
26     id.erase(ranges::find(id, t));
27     cut = min(cut, gt);
28     for (int j = 1; j <= n; ++j)
29         w[s][j] += w[t][j], w[j][s] += w[j][t];
30 }
31 return cut;
32 }

```

10 String

10.1 Rolling Hash

```

1 const ll C = 27;
2 inline int id(char c) {return c-'a'+1;}
3 struct RollingHash {
4     string s; int n; ll mod;
5     vector<ll> Cexp, hs;
6     RollingHash(string& _s, ll _mod):
7         s(_s), n((int)_s.size()), mod(_mod)
8     {
9         Cexp.assign(n, 0);
10        hs.assign(n, 0);
11        Cexp[0] = 1;
12        for (int i = 1; i < n; i++) {
13            Cexp[i] = Cexp[i-1] * C;
14            if (Cexp[i] >= mod) Cexp[i] %= mod;
15        }
16        hs[0] = id(s[0]);
17        for (int i = 1; i < n; i++) {
18            hs[i] = hs[i-1] * C + id(s[i]);
19            if (hs[i] >= mod) hs[i] %= mod;
20        }
21        inline ll query(int l, int r) {
22            ll res = hs[r] - (l ? hs[l-1] * Cexp[r-l+1] : 0);
23            res = (res % mod + mod) % mod;
24            return res;
25        }
26    };

```

10.2 KMP, Z Value

```

1 int n, m;
2 string s, p;
3 vector<int> f;
4 void build() {
5     f.clear(); f.resize(m, 0);
6     int ptr = 0; for (int i = 1; i < m; i++) {
7         while (ptr && p[i] != p[ptr]) ptr = f[ptr-1];
8         if (p[i] == p[ptr]) ptr++;
9         f[i] = ptr;
10    }
11    void init() {
12        cin >> s >> p;
13        n = (int)s.size();
14        m = (int)p.size();
15        build();
16    }
17    void solve() {
18        int ans = 0, pi = 0;
19        for (int si = 0; si < n; si++) {
20            while (pi && s[si] != p[pi]) pi = f[pi-1];
21            if (s[si] == p[pi]) pi++;
22            if (pi == m) ans++, pi = f[pi-1];
23        }
24        cout << ans << endl;
25    }
26    string is, it, s;
27    int n; vector<int> z;
28    void init() {
29        cin >> is >> it;
30        s = it+'0'+is;
31        n = (int)s.size();
32        z.resize(n, 0);
33    }
34    void solve() {
35        int ans = 0; z[0] = n;
36        for (int i = 1, l = 0, r = 0; i < n; i++) {
37            if (i <= r) z[i] = min(z[i-l], r-i+1);
38            while (i+z[i] < n && s[z[i]] == s[i+z[i]]) z[i]++;
39            if (i+z[i]-1 > r) l = i, r = i+z[i]-1;
40            if (z[i] == (int)it.size()) ans++;
41        }
42        cout << ans << endl;
43    }

```

10.3 Manacher

```

1 int n; string S, s;
2 vector<int> m;
3 void manacher() {
4     s.clear(); s.resize(2*n+1, '.');
5     for (int i = 0, j = 1; i < n; i++, j += 2) s[j] = S[i];
6     m.clear(); m.resize(2*n+1, 0);
7     // m[i] := max k such that s[i-k, i+k] is palindrome
8     int mx = 0, mxk = 0;
9     for (int i = 1; i < 2*n+1; i++) {

```

```

10     if (mx-(i-mx) >= 0) m[i] = min(m[mx-(i-mx)], mx+mxk-i);
11     while (0 <= i-m[i]-1 && i+m[i]+1 < 2*n+1 &&
12            s[i-m[i]-1] == s[i+m[i]+1]) m[i]++;
13     if (i+m[i] > mx+mxk) mx = i, mxk = m[i];
14 }
15 void init() { cin >> S; n = (int)S.size(); }
16 void solve() {
17     manacher();
18     int mx = 0, ptr = 0;
19     for (int i = 0; i < 2*n+1; i++) if (mx < m[i])
20         { mx = m[i]; ptr = i; }
21     for (int i = ptr-mx; i <= ptr+mx; i++)
22         if (s[i] != '.') cout << s[i];
23     cout << endl;

```

10.4 Suffix Array

```

1 #define F first
2 #define S second
3 struct SuffixArray { // don't forget s += "$";
4     int n; string s;
5     vector<int> suf, lcp, rk;
6     vector<int> cnt, pos;
7     vector<pair<pii, int>> buc[2];
8     void init(string _s) {
9         s = _s; n = (int)s.size();
10        // resize(n): suf, rk, cnt, pos, lcp, buc[0~1]
11    }
12    void radix_sort() {
13        for (int t : {0, 1}) {
14            fill(cnt.begin(), cnt.end(), 0);
15            for (auto& i : buc[t]) cnt[(t ? i.F.F :
16                i.F.S) ]++;
17            for (int i = 0; i < n; i++)
18                pos[i] = (!i ? 0 : pos[i-1] + cnt[i-1]);
19            for (auto& i : buc[t])
20                buc[t][i][pos[(t ? i.F.F : i.F.S) ]++] = i;
21        }
22        bool fill_suf() {
23            bool end = true;
24            for (int i = 0; i < n; i++) suf[i] = buc[0][i].S;
25            rk[suf[0]] = 0;
26            for (int i = 1; i < n; i++) {
27                int dif = (buc[0][i].F != buc[0][i-1].F);
28                end &= dif;
29                rk[suf[i]] = rk[suf[i-1]] + dif;
30            }
31            return end;
32        }
33        void sa() {
34            for (int i = 0; i < n; i++)
35                buc[0][i] = make_pair(make_pair(s[i], s[i]), i);
36            sort(buc[0].begin(), buc[0].end());
37            if (fill_suf()) return;
38            for (int k = 0; (1<<k) < n; k++) {
39                for (int i = 0; i < n; i++)
40                    buc[0][i] = make_pair(make_pair(rk[i], rk[(i
41                        + (1<<k) % n)]), i);
42                radix_sort();
43                if (fill_suf()) return;
44            }
45            void LCP() { int k = 0;
46                for (int i = 0; i < n-1; i++) {
47                    if (rk[i] == 0) continue;
48                    int pi = rk[i];
49                    int j = suf[pi-1];
50                    while (i+k < n && j+k < n && s[i+k] == s[j+k])
51                        k++;
52                    lcp[pi] = k;
53                    k = max(k-1, 0);
54                }
55            }
56        }
57    };
58    SuffixArray suffixarray;

```

10.5 Suffix Automaton

```

1 // any path start from root forms a substring of S
2 // occurrence of P: iff SAM can run on input word P
3 // number of different substring: ds[1]-1
4 // total length of all different substring: dsl[1]
5 // max/min length of state i: mx[i]/mx[mom[i]]+1
6 // assume a run on input word P end at state i:
7 // number of occurrences of P: cnt[i]
8 // first occurrence position of P: fp[i]-|P|+1
9 // all position: !clone nodes in dfs from i through rmom
10 const int MXM=1000010;
11 struct SAM{
12     int tot, root, lst, mom[MXM], mx[MXM]; // ind[MXM]
13     int nxt[MXM][33]; // cnt[MXM], ds[MXM], dsl[MXM], fp[MXM]
14     // bool v[MXM], clone[MXN]
15     int newNode(){
16         int res=++tot; fill(nxt[res], nxt[res]+33, 0);

```

```

17 mom[res]=mx[res]=0; // cnt=ds=ds1=fp=v=clone=0
18 return res;
19 }
20 void init(){ tot=0;root=newNode();lst=root; }
21 void push(int c){
22     int p=lst,np=newNode(); // cnt[np]=1,clone[np]=0
23     mx[np]=mx[p]+1; // fp[np]=mx[np]-1
24     for(;p&&nxt[p][c]==0;p=mom[p]) nxt[p][c]=np;
25     if(p==0) mom[np]=root;
26     else{
27         int q=nxt[p][c];
28         if(mx[p]+1==mx[q]) mom[np]=q;
29         else{
30             int nq=newNode(); // fp[nq]=fp[q],clone[nq]=1
31             mx[nq]=mx[p]+1;
32             for(int i=0;i<33;i++) nxt[nq][i]=nxt[q][i];
33             mom[nq]=mom[q]; mom[q]=nq; mom[np]=nq;
34             for(;p&&nxt[p][c]==q;p=mom[p]) nxt[p][c]=nq;
35         }
36     }
37     lst=np;
38 }
39 void calc(){
40     calc(root); iota(ind,ind+tot,1);
41     sort(ind,ind+tot,&[](int i,int j){return mx[i]<mx[j]});
42     for(int i=tot-1;i>=0;i--){
43         cnt[mom[ind[i]]]+=cnt[ind[i]];
44     }
45 }
46 void calc(int x){
47     v[x]=ds[x]=1;ds1[x]=0; // rmom[mom[x]].push_back(x);
48     for(int i=0;i<26;i++){
49         if(nxt[x][i]){
50             if(!v[nxt[x][i]]) calc(nxt[x][i]);
51             ds[x]+=ds[nxt[x][i]];
52             ds1[x]+=ds[nxt[x][i]]+ds1[nxt[x][i]];
53         }
54     }
55     void push(char *str){
56         for(int i=0;str[i];i++) push(str[i]-'a');
57     }
58 }
59 } sam;

```

10.6 Minimum Rotation

```

1 // lexicographically minimal rotation
2 // rotate(begin(s), begin(s)+minRotation(s), end(s))
3 int minRotation(string s) {
4     int a = 0, n = s.size(); s += s;
5     for(int b = 0; b < n; b++) for(int k = 0; k < n; k++) {
6         if(a + k == b || s[a + k] < s[b + k]) {
7             b += max(0, k - 1);
8             break; }
9         if(s[a + k] > s[b + k]) {
10            a = b;
11            break; }
12     }
13     return a; }

```

10.7 Aho Corasick

```

1 struct AAutomata{
2     struct Node{
3         int cnt;
4         Node *go[26], *fail, *dic;
5         Node() {
6             cnt = 0; fail = 0; dic = 0;
7             memset(go,0,sizeof(go));
8         }
9     } pool[1048576], *root;
10    int nMem;
11    Node* new_Node(){
12        pool[nMem] = Node();
13        return &pool[nMem++];
14    }
15    void init() { nMem = 0; root = new_Node(); }
16    void add(const string &str) { insert(root,str,0); }
17    void insert(Node *cur, const string &str, int pos){
18        for(int i=pos;i<str.size();i++){
19            if(!cur->go[str[i]-'a'])
20                cur->go[str[i]-'a'] = new_Node();
21            cur=cur->go[str[i]-'a'];
22        }
23        cur->cnt++;
24    }
25    void make_fail(){
26        queue<Node*> que;
27        que.push(root);
28        while (!que.empty()){
29            Node* fr=que.front(); que.pop();
30            for (int i=0; i<26; i++){
31                if (fr->go[i]){
32                    Node *ptr = fr->fail;
33                    while (ptr && !ptr->go[i]) ptr = ptr->fail;

```

```

34        fr->go[i]->fail=ptr=(ptr?ptr->go[i]:root);
35        fr->go[i]->dic=(ptr->cnt?ptr:ptr->dic);
36        que.push(fr->go[i]);
37    } } }
38}AC;

```

11 Geometry

11.1 Template

```

1 // Author: Gino
2 typedef long long T;
3 // typedef long double T;
4 const long double eps = 1e-8;
5
6 short sgn(T x) {
7     if (abs(x) < eps) return 0;
8     return x < 0 ? -1 : 1;
9 }
10
11 struct Pt {
12     T x, y;
13     Pt(T _x=0, T _y=0):x(_x), y(_y) {}
14     Pt operator+(Pt a) { return Pt(x+a.x, y+a.y); }
15     Pt operator-(Pt a) { return Pt(x-a.x, y-a.y); }
16     Pt operator*(T a) { return Pt(x*a, y*a); }
17     Pt operator/(T a) { return Pt(x/a, y/a); }
18     T operator*(Pt a) { return x*a.x + y*a.y; }
19     T operator^(Pt a) { return x*a.y - y*a.x; } // 不要打反
20     bool operator<(Pt a)
21     { return x < a.x || (x == a.x && y < a.y); }
22     // return sgn(x-a.x) < 0 || (sgn(x-a.x) == 0 && sgn(y-a.y) < 0); }
23     bool operator==(Pt a)
24     { return sgn(x-a.x) == 0 && sgn(y-a.y) == 0; }
25 };
26
27 Pt mv(Pt a, Pt b) { return b-a; }
28 T len2(Pt a) { return a*a; }
29 T dis2(Pt a, Pt b) { return len2(b-a); }
30
31 short ori(Pt a, Pt b) { return ((a^b)>0) - ((a^b)<0); }
32 bool onseg(Pt p, Pt l1, Pt l2) {
33     Pt a = mv(p, l1), b = mv(p, l2);
34     return ((a^b) == 0) && ((a^b) <= 0);
35 }

```

11.2 Basic Operations

```

1 // Author: Gino
2 // Function: Check if a point P sits in a polygon (doesn't
3 // have to be convex hull)
4 // 0 = Bound, 1 = In, -1 = Out
5 short inPoly(Pt p) {
6     for (int i = 0; i < n; i++)
7         if (onseg(p, E[i], E[(i+1)%n])) return 0;
8     int cnt = 0;
9     for (int i = 0; i < n; i++)
10        if (banana(p, Pt(p.x+1, p.y+2e9), E[i], E[(i+1)%n]))
11            cnt ^= 1;
12    return (cnt ? 1 : -1);
13 }

```

```

1 // Author: Gino
2 int ud(Pt a) { // up or down half plane
3     if (a.y > 0) return 0;
4     if (a.y < 0) return 1;
5     return (a.x >= 0 ? 0 : 1);
6 }
7
8 sort(ALL(E), [&](const Pt& a, const Pt& b){
9     if (ud(a) != ud(b)) return ud(a) < ud(b);
10    return (a^b) > 0;
11 });

```

```

1 // Author: Gino
2 // Function: check if (p1--p2) (q1--q2) banana
3 inline bool banana(Pt p1, Pt p2, Pt q1, Pt q2) {
4     if (onseg(p1, q1, q2) || onseg(p2, q1, q2) ||
5         onseg(q1, p1, p2) || onseg(q2, p1, p2)) {
6         return true;
7     }
8     Pt p = mv(p1, p2), q = mv(q1, q2);
9     return (ori(p, mv(p1, q1)) * ori(p, mv(p1, q2)) < 0 &&
10         ori(q, mv(q1, p1)) * ori(q, mv(q1, p2)) < 0);
11 }

```

```

1 // Author: Gino
2 // T: long double
3 Pt bananaPoint(Pt p1, Pt p2, Pt q1, Pt q2) {
4     if (onseg(q1, p1, p2)) return q1;
5     if (onseg(q2, p1, p2)) return q2;
6     if (onseg(p1, q1, q2)) return p1;
7     if (onseg(p2, q1, q2)) return p2;

```

```

8 double s = abs(mv(p1, p2) ^ mv(p1, q1));
9 double t = abs(mv(p1, p2) ^ mv(p1, q2));
10 return q2 * (s/(s+t)) + q1 * (t/(s+t));
11 }

1 // Author: Gino
2 vector<Pt> hull;
3 void convexHull() {
4 hull.clear(); sort(E.begin(), E.end());
5 for (int t : {0, 1}) {
6     int b = (int)hull.size();
7     for (auto& ei : E) {
8         while ((int)hull.size() - b >= 2 &&
9             ori(mv(hull[(int)hull.size()-2],
10                 mv(hull[(int)hull.size()-1], ei)) == -1)
11         {
12             hull.pop_back();
13         }
14         hull.emplace_back(ei);
15     }
16     hull.pop_back();
17     reverse(E.begin(), E.end());
18 } }

```

```

1 // Author: Gino
2 // Function: Return doubled area of a polygon
3 T dbarea(vector<Pt>& e) {
4 ll res = 0;
5 for (int i = 0; i < (int)e.size(); i++)
6     res += e[i]^e[(i+1)%SZ(e)];
7 return abs(res);
8 }

```

11.3 Lower Concave Hull

```

1 // Author: Unknown
2 struct Line {
3     mutable ll m, b, p;
4     bool operator<(const Line& o) const { return m < o.m; }
5     bool operator<(ll x) const { return p < x; }
6 };
7
8 struct LineContainer : multiset<Line, less<>> {
9     // (for doubles, use inf = 1/.0, div(a,b) = a/b)
10    const ll inf = LLONG_MAX;
11    ll div(ll a, ll b) { // floored division
12        return a / b - ((a ^ b) < 0 && a % b); }
13    bool isect(iterator x, iterator y) {
14        if (y == end()) { x->p = inf; return false; }
15        if (x->m == y->m) x->p = x->b > y->b ? inf : -inf;
16        else x->p = div(y->b - x->b, x->m - y->m);
17        return x->p >= y->p;
18    }
19    void add(ll m, ll b) {
20        auto z = insert({m, b, 0}), y = z++, x = y;
21        while (isect(y, z)) z = erase(z);
22        if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
23        while ((y = x) != begin() && (--x)->p >= y->p)
24            isect(x, erase(y));
25    }
26    ll query(ll x) {
27        assert(!empty());
28        auto l = *lower_bound(x);
29        return l.m * x + l.b;
30    }
31 };

```

11.4 Pick's Theorem

Consider a polygon which vertices are all lattice points.
 Let i = number of points inside the polygon.
 Let b = number of points on the boundary of the polygon.
 Then we have the following formula:

$$\text{Area} = i + b/2 - 1$$

11.5 Minimum Enclosing Circle

```

1 // Author: Gino
2 // Function: Find Min Enclosing Circle using Randomized O(n)
3 // Algorithm
4 Pt circumcenter(Pt A, Pt B, Pt C) {
5 // a1(x-A.x) + b1(y-A.y) = c1
6 // a2(x-A.x) + b2(y-A.y) = c2
7 // solve using Cramer's rule
8 T a1 = B.x-A.x, b1 = B.y-A.y, c1 = dis2(A, B)/2.0;
9 T a2 = C.x-A.x, b2 = C.y-A.y, c2 = dis2(A, C)/2.0;
10 T D = Pt(a1, b1) ^ Pt(a2, b2);
11 T Dx = Pt(c1, b1) ^ Pt(c2, b2);
12 T Dy = Pt(a1, c1) ^ Pt(a2, c2);
13 if (D == 0) return Pt(-INF, -INF);

```

```

13 return A + Pt(Dx/D, Dy/D);
14 }
15 Pt center; T r2;
16
17 void minEncloseCircle() {
18 mt19937
19     gen(chrono::steady_clock::now().time_since_epoch().count());
20 shuffle(ALL(E), gen);
21 center = E[0], r2 = 0;
22
23 for (int i = 0; i < n; i++) {
24     if (dis2(center, E[i]) <= r2) continue;
25     center = E[i], r2 = 0;
26     for (int j = 0; j < i; j++) {
27         if (dis2(center, E[j]) <= r2) continue;
28         center = (E[i] + E[j]) / 2.0;
29         r2 = dis2(center, E[i]);
30         for (int k = 0; k < j; k++) {
31             if (dis2(center, E[k]) <= r2) continue;
32             center = circumcenter(E[i], E[j], E[k]);
33             r2 = dis2(center, E[i]);
34         }
35     }
36 }

```

11.6 PolyUnion

```

1 // Author: Unknown
2 struct PY {
3     int n; Pt pt[5]; double area;
4     Pt& operator[](const int x) { return pt[x]; }
5     void init() { //n,pt[0~n-1] must be filled
6         area=pt[n-1]^pt[0];
7         for(int i=0;i<n-1;i++) area+=pt[i]^pt[i+1];
8         if((area/=2)<0)reverse(pt,pt+n),area=-area;
9     }
10 };
11 PY py[500]; pair<double,int> c[5000];
12 inline double segP(Pt &p,Pt &p1,Pt &p2){
13     if(dcmp(p1.x-p2.x)==0) return (p.y-p1.y)/(p2.y-p1.y);
14     return (p.x-p1.x)/(p2.x-p1.x);
15 }
16 double polyUnion(int n){ //py[0~n-1] must be filled
17     int i,j,ii,jj,ta,tb,r,d; double z,w,s,sum=0,tc,td;
18     for(i=0;i<n;i++) py[i][py[i].n]=py[i][0];
19     for(i=0;i<n;i++){
20         for(ii=0;ii<py[i].n;ii++){
21             r=0;
22             c[r++]=make_pair(0.0,0); c[r++]=make_pair(1.0,0);
23             for(j=0;j<n;j++){
24                 if(i==j) continue;
25                 for(jj=0;jj<py[j].n;jj++){
26                     ta=dcmp(tri(py[i][ii],py[i][ii+1],py[j][jj]));
27                     tb=dcmp(tri(py[i][ii+1],py[i][ii+1],py[j][jj+1]));
28                     if(ta==0 && tb==0){
29                         if((py[j][jj+1]-py[j][jj])*(py[i][ii+1]-py[i][ii])>0 && j<i){
30                             c[r++]=make_pair(segP(py[j][jj],py[i][ii],py[i][ii+1]),1);
31                             c[r++]=make_pair(segP(py[j][jj+1],py[i][ii+1],py[i][ii+1]),-1);
32                         }
33                     } else if(ta>0 && tb<0){
34                         tc=tri(py[j][jj],py[j][jj+1],py[i][ii]);
35                         td=tri(py[j][jj],py[j][jj+1],py[i][ii+1]);
36                         c[r++]=make_pair(tc/(tc+td),1);
37                     } else if(ta<0 && tb>0){
38                         tc=tri(py[j][jj],py[j][jj+1],py[i][ii]);
39                         td=tri(py[j][jj],py[j][jj+1],py[i][ii+1]);
40                         c[r++]=make_pair(tc/(tc+td),-1);
41                     }
42                 }
43             }
44             sort(c,c+r);
45             z=min(max(c[0].first,0.0),1.0); d=c[0].second; s=0;
46             for(j=1;j<r;j++){
47                 w=min(max(c[j].first,0.0),1.0);
48                 if(!d) s+=w-z;
49                 d+=c[j].second; z=w;
50             }
51             sum+=(py[i][ii]^py[i][ii+1])*s;
52         }
53     }
54     return sum/2;
55 }

```

11.7 Minkowski Sum

```

1 // Author: Unknown
2 /* convex hull Minkowski Sum*/
3 #define INF 1000000000000000LL
4 int pos(const Pt& tp){
5     if(tp.Y == 0) return tp.X > 0 ? 0 : 1;
6     return tp.Y > 0 ? 0 : 1;

```

```

7 }
8 #define N 300030
9 Pt pt[ N ], qt[ N ], rt[ N ];
10 LL Lx,Rx;
11 int dn,un;
12 inline bool cmp( Pt a, Pt b ){
13     int pa=pos( a ),pb=pos( b );
14     if(pa==pb) return (a^b)>0;
15     return pa<pb;
16 }
17 int minkowskiSum(int n,int m){
18     int i,j,r,p,q,fi,fj;
19     for(i=1,p=0;i<n;i++){
20         if( pt[i].Y<pt[p].Y ||
21            (pt[i].Y==pt[p].Y && pt[i].X<pt[p].X) ) p=i; }
22     for(i=1,q=0;i<m;i++){
23         if( qt[i].Y<qt[q].Y ||
24            (qt[i].Y==qt[q].Y && qt[i].X<qt[q].X) ) q=i; }
25     rt[0]=pt[p]+qt[q];
26     r=1; i=p; j=q; fi=fj=0;
27     while(1){
28         if((fj&&j==q) ||
29            ( (!fi||i==p) &&
30              cmp(pt[(p+1)%n]-pt[p],qt[(q+1)%m]-qt[q]) ) ){
31             rt[r]=rt[r-1]+pt[(p+1)%n]-pt[p];
32             p=(p+1)%n;
33             fi=1;
34         }else{
35             rt[r]=rt[r-1]+qt[(q+1)%m]-qt[q];
36             q=(q+1)%m;
37             fj=1;
38         }
39         if(r<=1 || ((rt[r]-rt[r-1])^(rt[r-1]-rt[r-2]))!=0) r++;
40         else rt[r-1]=rt[r];
41         if(i==p && j==q) break;
42     }
43     return r-1;
44 }
45 void initInConvex(int n){
46     int i,p,q;
47     LL Ly,Ry;
48     Lx=INF; Rx=-INF;
49     for(i=0;i<n;i++){
50         if(pt[i].X<Lx) Lx=pt[i].X;
51         if(pt[i].X>Rx) Rx=pt[i].X;
52     }
53     Ly=Ry=INF;
54     for(i=0;i<n;i++){
55         if(pt[i].X==Lx && pt[i].Y<Ly){ Ly=pt[i].Y; p=i; }
56         if(pt[i].X==Rx && pt[i].Y<Ry){ Ry=pt[i].Y; q=i; }
57     }
58     for(dn=0,i=p;i!=q;i=(i+1)%n){ qt[dn++]=pt[i]; }
59     qt[dn]=pt[q]; Ly=Ry=-INF;
60     for(i=0;i<n;i++){
61         if(pt[i].X==Lx && pt[i].Y>Ly){ Ly=pt[i].Y; p=i; }
62         if(pt[i].X==Rx && pt[i].Y>Ry){ Ry=pt[i].Y; q=i; }
63     }
64     for(un=0,i=p;i!=q;i=(i+n-1)%n){ rt[un++]=pt[i]; }
65     rt[un]=pt[q];
66 }
67 inline int inConvex(Pt p){
68     int L,R,M;
69     if(p.X<Lx || p.X>Rx) return 0;
70     L=0;R=dn;
71     while(L<R-1){ M=(L+R)/2;
72         if(p.X<qt[M].X) R=M; else L=M; }
73     if(tri(qt[L],qt[R],p)<0) return 0;
74     L=0;R=un;
75     while(L<R-1){ M=(L+R)/2;
76         if(p.X<rt[M].X) R=M; else L=M; }
77     if(tri(rt[L],rt[R],p)>0) return 0;
78     return 1;
79 }
80 int main(){
81     int n,m,i;
82     Pt p;
83     scanf("%d",&n);
84     for(i=0;i<n;i++) scanf("%lld%lld",&pt[i].X,&pt[i].Y);
85     scanf("%d",&m);
86     for(i=0;i<m;i++) scanf("%lld%lld",&qt[i].X,&qt[i].Y);
87     n=minkowskiSum(n,m);
88     for(i=0;i<n;i++) pt[i]=rt[i];
89     scanf("%d",&m);
90     for(i=0;i<m;i++) scanf("%lld%lld",&qt[i].X,&qt[i].Y);
91     n=minkowskiSum(n,m);
92     for(i=0;i<n;i++) pt[i]=rt[i];
93     initInConvex(n);
94     scanf("%d",&m);

```

```

95     for(i=0;i<m;i++){
96         scanf("%lld%lld",&p.X,&p.Y);
97         p.X*=3; p.Y*=3;
98         puts(inConvex(p)?"YES":"NO");
99     }
100 }

```

12 Number Theory

12.1 Mod Sum

```

1 /**
2  * Author: Simon Lindholm
3  * Date: 2015-06-23
4  * License: CC0
5  * Source: own work
6  * Description: Sums of mod'ed arithmetic progressions.
7  *
8  * \texttt{modsum(to, c, k, m)} =  $\sum_{i=0}^{\text{to}-1} (ki+c) \% m$ .
9  * \texttt{divsum} is similar but for floored division.
10 * Time:  $\log(m)$ , with a large constant.
11 * Status: Tested for all  $|k|, |c|, \text{to}, m \leq 50$ , and on
12   kattis:aladin
13 */
14 #pragma once
15 typedef unsigned long long ull;
16 ull sumsq(ull to) { return to / 2 * ((to-1) | 1); }
17 /// ^ written in a weird way to deal with overflows
18   correctly
19 ull divsum(ull to, ull c, ull k, ull m) {
20     ull res = k / m * sumsq(to) + c / m * to;
21     k %= m; c %= m;
22     if (!k) return res;
23     ull to2 = (to * k + c) / m;
24     return res + (to - 1) * to2 - divsum(to2, m-1 - c, m, k);
25 }
26 ll modsum(ull to, ll c, ll k, ll m) {
27     c = ((c % m) + m) % m;
28     k = ((k % m) + m) % m;
29     return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
30 }
31 }

```

12.2 Extended Lucas Theorem

```

1 /**
2  * Author: koukanni
3  * Date: 2025
4  * License: CC0
5  * Source:
6  * Description: Computes  $C(n, m) \bmod P$  where  $P$  is not
7   necessarily prime
8  * Functions
9  * exgcd(a, b, x, y): Extended Euclidean algorithm
10 * inv(a, p): Modular inverse of a modulo p
11 * qpow(a, b, p): Fast modular exponentiation
12 * CRT(n, a, m): Chinese Remainder Theorem
13 * calc(n, x, P): Helper function for computing factorial
14   with prime power
15 * multilucas(n, m, x, P): Lucas theorem for prime power
16   modulus
17 * exlucas(n, m, P): Main function - Extended Lucas theorem
18 * usage: exlucas(n, m, P) returns  $C(n, m) \bmod P$ 
19 * Time:  $O(P \log(P))$ 
20 * Status: tested
21 */
22 typedef long long ll;
23 void exgcd(ll a, ll b, ll& x, ll& y) {
24     if (!b)
25         x = 1, y = 0;
26     else
27         exgcd(b, a % b, y, x), y -= a / b * x;
28 }
29 ll inv(ll a, ll p) {
30     ll x, y;
31     exgcd(a, p, x, y);
32     return (x + p) % p;
33 }
34 ll qpow(ll a, ll b, ll p) {
35     ll r = 1;
36     for (; b >= 1; a = a * a % p)
37         if (b & 1) r = r * a % p;
38     return r;
39 }
40 ll CRT(int n, ll* a, ll* m) {
41     ll M = 1, p = 0;
42     for (int i = 1; i <= n; i++) M = M * m[i];
43     for (int i = 1; i <= n; i++) {
44         ll w = M / m[i], x, y;

```



```

42     exgcd(w, m[i], x, y);
43     p = (p + a[i] * w * x % M) % M;
44 }
45 return (p % M + M) % M;
46 }
47 ll calc(ll n, ll x, ll P) {
48     if (!n) return 1;
49     ll s = 1;
50     for (ll i = 1; i <= P; i++)
51         if (i % x) s = s * i % P;
52     s = qpow(s, n / P, P);
53     for (ll i = n / P * P + 1; i <= n; i++)
54         if (i % x) s = i % P * s % P;
55     return s * calc(n / x, x, P) % P;
56 }
57 ll multilucas(ll n, ll m, ll x, ll P) {
58     if (n < m) return 0;
59     int cnt = 0;
60     for (ll i = n; i; i /= x) cnt += i / x;
61     for (ll i = m; i; i /= x) cnt -= i / x;
62     for (ll i = n - m; i; i /= x) cnt -= i / x;
63     return qpow(x, cnt, P) % P * calc(n, x, P) % P *
        inv(calc(m, x, P), P) % P *
        inv(calc(n - m, x, P), P) % P;
64 }
65 }
66 ll exlucas(ll n, ll m, ll P) {
67     int cnt = 0;
68     ll p[20], a[20];
69     for (ll i = 2; i * i <= P; i++) {
70         if (P % i == 0) {
71             p[++cnt] = i;
72             while (P % i == 0) p[cnt] = p[cnt] * i, P /= i;
73             a[cnt] = multilucas(n, m, i, p[cnt]);
74         }
75     }
76     if (P > 1) p[++cnt] = P, a[cnt] = multilucas(n, m, P, P);
77     return CRT(cnt, a, p);
78 }

```

12.3 Prime Sieve and Defactor

```

1 // Author: Gino
2 const int maxc = 1e6 + 1;
3 vector<int> lpf;
4 vector<int> prime;
5
6 void seive() {
7     prime.clear();
8     lpf.resize(maxc, 1);
9     for (int i = 2; i < maxc; i++) {
10         if (lpf[i] == 1) {
11             lpf[i] = i;
12             prime.emplace_back(i);
13         }
14         for (auto& j : prime) {
15             if (i * j >= maxc) break;
16             lpf[i * j] = j;
17             if (j == lpf[i]) break;
18         }
19     }
20     vector<pii> fac;
21     void defactor(int u) {
22         fac.clear();
23         while (u > 1) {
24             int d = lpf[u];
25             fac.emplace_back(make_pair(d, 0));
26             while (u % d == 0) {
27                 u /= d;
28                 fac.back().second++;
29             }
30         }
31     }
32 }

```

12.4 Harmonic Series

```

1 // Author: Gino
2 //  $O(n \log n)$ 
3 for (int i = 1; i <= n; i++) {
4     for (int j = i; j <= n; j += i) {
5         //  $O(1)$  code
6     }
7 }
8
9 // PIE
10 // given array a[0], a[1], ..., a[n - 1]
11 // calculate dp[x] = number of pairs (a[i], a[j]) such that
12 // gcd(a[i], a[j]) = x // (i < j)
13 //
14 // idea: let mc(x) = # of y s.t. x|y
15 // f(x) = # of pairs s.t. gcd(a[i], a[j]) >= x
16 // f(x) = C(mc(x), 2)
17 // dp[x] = f(x) - sum(dp[y], x < y and x|y)
18 const int maxc = 1e6;
19 vector<int> cnt(maxc + 1, 0), dp(maxc + 1, 0);
20 for (int i = 0; i < n; i++)

```

```

21     cnt[a[i]]++;
22
23 for (int x = maxc; x >= 1; x--) {
24     ll cnt_mul = 0; // number of multiples of x
25     for (int y = x; y <= maxc; y += x)
26         cnt_mul += cnt[y];
27
28     dp[x] = cnt_mul * (cnt_mul - 1) / 2; // number of pairs
29     // that are divisible by x
30     for (int y = x + x; y <= maxc; y += x)
31         dp[x] -= dp[y]; // PIE: subtract all dp[y] for y > x and x|y

```

12.5 Count Number of Divisors

```

1 // Author: Gino
2 // Function: Count the number of divisors for all x <= 10^6
3 // using harmonic series
4 const int maxc = 1e6;
5 vector<int> facs;
6
7 void find_all_divisors() {
8     facs.clear(); facs.resize(maxc + 1, 0);
9     for (int x = 1; x <= maxc; x++) {
10         for (int y = x; y <= maxc; y += x) {
11             facs[y]++;
12         }
13     }

```

12.6 數論分塊

```

1 // Author: Gino
2 /*
3 n = 17
4 i: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17
5 n/i: 17 8 5 4 3 2 2 2 1 1 1 1 1 1 1 1 1
6           ^       ^
7           L(2)   R(2)
8
9 L(x) := left bound for n/i = x
10 R(x) := right bound for n/i = x
11
12 ===== FORMULA =====
13 >>> R = n / (n/L) <<<
14 =====
15
16 Example: L(2) = 6
17            R(2) = 17 / (17 / 6)
18                = 17 / 2
19                = 8
20 */
21 // ===== CODE =====
22
23 for (ll l = 1, r = 1, q = n; l <= n; l = r + 1) {
24     q = n/l;
25     r = n/q;
26     // Process your code here
27 }
28 // q, l, r: 17 1 1
29 // q, l, r: 8 2 2
30 // q, l, r: 5 3 3
31 // q, l, r: 4 4 4
32 // q, l, r: 3 5 5
33 // q, l, r: 2 6 8
34 // q, l, r: 1 9 17

```

12.7 Pollard's rho

```

1 // Author: Unknown
2 // Function: Find a non-trivial factor of a big number in
3 //  $O(n^{1/4} \log^2(n))$ 
4 ll find_factor(ll number) {
5     __int128 x = 2;
6     for (__int128 cycle = 1; ; cycle++) {
7         __int128 y = x;
8         for (int i = 0; i < (1 << cycle); i++) {
9             x = (x * x + 1) % number;
10            __int128 factor = __gcd(x - y, number);
11            if (factor > 1)
12                return factor;
13        }
14    }
15 }
16
17 # Author: Unknown
18 # Function: Find a non-trivial factor of a big number in
19 //  $O(n^{1/4} \log^2(n))$ 
20 from itertools import count
21 from math import gcd
22 from sys import stdin
23
24

```

```

7 for s in stdin:
8     number, x = int(s), 2
9     brk = False
10    for cycle in count(1):
11        y = x
12        if brk:
13            break
14        for i in range(1 << cycle):
15            x = (x * x + 1) % number
16            factor = gcd(x - y, number)
17            if factor > 1:
18                print(factor)
19                brk = True
20            break

```

12.8 Miller Rabin

```

1 // Author: Unknown, Modified by Gino
2 // Function: Check if a number is a prime in  $O(100 * \log^2(n))$ 
3 // miller_rabin(): return 1 if prime, 0 otherwise
4 inline ll mul(ll x, ll y, ll mod) {
5     return (__int128)(x) * y % mod;
6 }
7 ll mypow(ll a, ll b, ll mod) {
8     ll r = 1;
9     while (b > 0) {
10        if (b & 1) r = mul(r, a, mod);
11        a = mul(a, a, mod);
12        b >>= 1;
13    }
14    return r;
15 }
16 bool witness(ll a, ll n, ll u, int t){
17     ll x = mypow(a, u, n);
18     for(int i = 0; i < t; i++) {
19         ll nx = mul(x, x, n);
20         if (nx == 1 && x != 1 && x != n-1) return true;
21         x = nx;
22     }
23     return x != 1;
24 }
25 bool miller_rabin(ll n) {
26     // if n >= 3,474,749,660,383
27     // change {2, 3, 5, 7, 11, 13} to
28     // {2, 325, 9375, 28178, 450775, 9780504, 1795265022}
29     if (n < 2) return false;
30     if (!(n & 1)) return n == 2;
31     ll u = n - 1; int t = 0;
32     while(!(u & 1)) u >>= 1, t++;
33     for (ll a : {2, 3, 5, 7, 11, 13}) {
34         if (a % n == 0) continue;
35         if (witness(a, n, u, t)) return false;
36     }
37     return true;
38 }
39 // bases that make sure no pseudoprimes flee from test
40 // if WA, replace randll(n - 1) with these bases:
41 // n < 4,759,123,141          3 : 2, 7, 61
42 // n < 1,122,004,669,633    4 : 2, 13, 23, 1662803
43 // n < 3,474,749,660,383    6 : pirmses <= 13
44 // n < 2^64                7 :
45 // 2, 325, 9375, 28178, 450775, 9780504, 1795265022

```

12.9 Discrete Log

```

1 // Author: ckiseki
2 // find  $x^? = y \pmod{M}$ 
3 //  $O(\sqrt{M})$ 
4 template<typename Int>
5 Int BSGS(Int x, Int y, Int M) {
6     //  $x^? \equiv y \pmod{M}$ 
7     Int t = 1, c = y, g = 1;
8     for (Int M_ = M; M_ > 0; M_ >>= 1) g = g * x % M;
9     for (g = gcd(g, M); t % g != 0; ++c) {
10        if (t == y) return c;
11        t = t * x % M;
12    }
13    if (y % g != 0) return -1;
14    t /= g, y /= g, M /= g;
15    Int h = 0, gs = 1;
16    for (; h * h < M; ++h) gs = gs * x % M;
17    unordered_map<Int, Int> bs;
18    for (Int s = 0; s < h; bs[y] = ++s) y = y * x % M;
19    for (Int s = 0; s < M; s += h) {
20        t = t * gs % M;
21        if (bs.count(t)) return c + s + h - bs[t];
22    }
23    return -1;
24 }

```

12.10 Discrete Sqrt

```

1 // Author: ckiseki
2 // find solution for  $x^2 = n \pmod{P}$ 
3 //  $O(\log^2 P)$ 
4 int get_root(int n, int P) { // ensure  $0 \leq n < P$ 
5     if (P == 2 or n == 0) return n;
6     auto check = [&](ll x) {
7         return modpow(int(x), (P - 1) / 2, P);
8     };
9     if (check(n) != 1) return -1;
10    mt19937 rnd(7122); ll z = 1, w;
11    while (check(w = (z * z - n + P) % P) != P - 1)
12        z = rnd() % P;
13    const auto M = [P, w](auto &u, auto &v) {
14        auto [a, b] = u; auto [c, d] = v;
15        return make_pair((a * c + b * d % P * w) % P,
16            (a * d + b * c) % P);
17    };
18    pair<ll, ll> r(1, 0), e(z, 1);
19    for (int q = (P + 1) / 2; q >= 1; e = M(e, e))
20        if (q & 1) r = M(r, e);
21    return int(r.first); // sqrt(n) mod P where P is prime

```

12.11 Fast Power

Note: $a^n \equiv a^{(n \bmod (p-1))} \pmod{p}$

12.12 Extend GCD

```

1 // Author: Gino
2 // [Usage]
3 // bezout(a, b, c):
4 //     find solution to  $ax + by = c$ 
5 //     return {-LINF, -LINF} if no solution
6 // inv(a, p):
7 //     find modulo inverse of a under p
8 //     return -1 if not exist
9 // CRT(vector<ll>& a, vector<ll>& m)
10 //     find a solution pair (x, mod) satisfies all  $x = a[i] \pmod{m[i]}$ 
11 //     return {-LINF, -LINF} if no solution
12
13 const ll LINF = 4e18;
14 typedef pair<ll, ll> pll;
15 template<typename T1, typename T2>
16 T1 chmod(T1 a, T2 m) {
17     return (a % m + m) % m;
18 }
19
20 ll GCD;
21 pll extgcd(ll a, ll b) {
22     if (b == 0) {
23         GCD = a;
24         return pll{1, 0};
25     }
26     pll ans = extgcd(b, a % b);
27     return pll{ans.second, ans.first - a/b * ans.second};
28 }
29 pll bezout(ll a, ll b, ll c) {
30     bool negx = (a < 0), negy = (b < 0);
31     pll ans = extgcd(abs(a), abs(b));
32     if (c % GCD != 0) return pll{-LINF, -LINF};
33     return pll{ans.first * c/GCD * (negx ? -1 : 1),
34         ans.second * c/GCD * (negy ? -1 : 1)};
35 }
36 ll inv(ll a, ll p) {
37     if (p == 1) return -1;
38     pll ans = bezout(a % p, -p, 1);
39     if (ans == pll{-LINF, -LINF}) return -1;
40     return chmod(ans.first, p);
41 }
42 pll CRT(vector<ll>& a, vector<ll>& m) {
43     for (int i = 0; i < (int)a.size(); i++)
44         a[i] = chmod(a[i], m[i]);
45
46     ll x = a[0], mod = m[0];
47     for (int i = 1; i < (int)a.size(); i++) {
48         pll sol = bezout(mod, m[i], a[i] - x);
49         if (sol.first == -LINF) return pll{-LINF, -LINF};
50
51         // prevent long long overflow
52         ll p = chmod(sol.first, m[i] / GCD);
53         ll lcm = mod / GCD * m[i];
54         x = chmod((__int128)p * mod + x, lcm);
55         mod = lcm;
56     }
57     return pll{x, mod};
58 }

```

12.13 Mu + Phi

1 // Author: Gino

```

2 const int maxn = 1e6 + 5;
3 ll f[maxn];
4 vector<int> lpf, prime;
5 void build() {
6     lpf.clear(); lpf.resize(maxn, 1);
7     prime.clear();
8     f[1] = ...; /* mu[1] = 1, phi[1] = 1 */
9     for (int i = 2; i < maxn; i++) {
10         if (lpf[i] == 1) {
11             lpf[i] = i; prime.emplace_back(i);
12             f[i] = ...; /* mu[i] = 1, phi[i] = i-1 */
13         }
14         for (auto& j : prime) {
15             if (i*j >= maxn) break;
16             lpf[i*j] = j;
17             if (i % j == 0) f[i*j] = ...; /* 0, phi[i]*j */
18             else f[i*j] = ...; /* -mu[i], phi[i]*phi[j] */
19             if (j >= lpf[i]) break;
20         }
21     }
22 }

```

12.14 Other Formulas

- Pisano Period: 任何線性遞迴 (比如費氏數列) 模任何一個數字 M 都會循環, 找循環節 $\pi(M)$ 先質因數分解 $M = \prod p_i^{e_i}$, 然後 $\pi(M) = \text{lcm}(\pi(p_i^{e_i}))$,
- Inversion: $aa^{-1} \equiv 1 \pmod{m}$. a^{-1} exists iff $\gcd(a, m) = 1$.
- Linear inversion: $a^{-1} \equiv (m - \lfloor \frac{m}{a} \rfloor) \times (m \bmod a)^{-1} \pmod{m}$
- Fermat's little theorem: $a^p \equiv a \pmod{p}$ if p is prime.
- Euler function: $\varphi(n) = n \prod_{p|n} \frac{p-1}{p}$
- Euler theorem: $a^{\varphi(n)} \equiv 1 \pmod{n}$ if $\gcd(a, n) = 1$. If a, n are not coprime: 質因數分解 $n = \prod p_i^{e_i}$, 對每個 $p_i^{e_i}$ 分開看他們跟 a 是否互質 (互質: Fermat / 不互質: 夠大的指數會直接削成 0), 最後用 CRT 合併.
- Extended Euclidean algorithm: $ax + by = \gcd(a, b) = \gcd(b, a \bmod b) = \gcd(b, a - \lfloor \frac{a}{b} \rfloor b) = bx_1 + (a - \lfloor \frac{a}{b} \rfloor b)y_1 = ay_1 + b(x_1 - \lfloor \frac{a}{b} \rfloor y_1)$
- Divisor function: $\sigma_x(n) = \sum_{d|n} d^x$. $n = \prod_{i=1}^r p_i^{a_i}$.
 $\sigma_x(n) = \prod_{i=1}^r \frac{p_i^{(a_i+1)x} - 1}{p_i^x - 1}$ if $x \neq 0$. $\sigma_0(n) = \prod_{i=1}^r (a_i + 1)$.
- Chinese remainder theorem (Coprime Moduli):
 $x \equiv a_i \pmod{m_i}$.
 $M = \prod m_i$. $M_i = \frac{M}{m_i}$. $t_i = M_i^{-1}$.
 $x = kM + \sum a_i t_i M_i$, $k \in \mathbb{Z}$.
- Chinese remainder theorem:
 $x \equiv a_1 \pmod{m_1}, x \equiv a_2 \pmod{m_2} \Rightarrow x = m_1 p + a_1 = m_2 q + a_2 \Rightarrow m_1 p - m_2 q = a_2 - a_1$
Solve for (p, q) using ExtGCD.
 $x \equiv m_1 p + a_1 \equiv m_2 q + a_2 \pmod{\text{lcm}(m_1, m_2)}$
- Avoiding Overflow: $ca \bmod cb = c(a \bmod b)$
- Dirichlet Convolution: $(f * g)(n) = \sum_{d|n} f(d)g(\frac{n}{d})$
- Important Multiplicative Functions + Properties:
 - $\varepsilon(n) = [n = 1]$
 - $1(n) = 1$
 - $id(n) = n$
 - $\mu(n) = 0$ if n has squared prime factor
 - $\mu(n) = (-1)^k$ if $n = p_1 p_2 \dots p_k$
 - $\varepsilon = \mu * 1$
 - $\varphi = \mu * id$
 - $[n = 1] = \sum_{d|n} \mu(d)$
 - $[gcd = 1] = \sum_{d|gcd} \mu(d)$
- Möbius inversion: $f = g * 1 \Leftrightarrow g = f * \mu$

12.15 Polynomial

```

1 // Author: Gino
2 // Preparation: first set_mod(mod, g), then init_ntt()
3 // everytime you change modulus, you have to call
  init_ntt() again
4 //
5 // [Usage]
6 // polynomial: vector<ll> a, b
7 // negation: -a
8 // add/subtract: a += b, a -= b
9 // convolution: a *= b
10 // in-place modulo: mod(a, b)
11 // in-place inversion under mod x^N: inv(ia, N)
12

```

```

13
14 const int maxk = 20;
15 const int maxn = 1<<maxk;
16
17 using u64 = unsigned long long;
18 using u128 = __uint128_t;
19
20 int g;
21 u64 MOD;
22 u64 BARRETT_IM; // [ 2^64 / MOD ]
23
24 inline void set_mod(u64 m, int _g) {
25     g = _g;
26     MOD = m;
27     BARRETT_IM = (u128(1) << 64) / m;
28 }
29 inline u64 chmod(u128 x) {
30     u64 q = (u64)((x * BARRETT_IM) >> 64);
31     u64 r = (u64)(x - (u128)q * MOD);
32     if (r >= MOD) r -= MOD;
33     return r;
34 }
35 inline u64 mmul(u64 a, u64 b) {
36     return chmod((u128)a * b);
37 }
38 ll pw(ll a, ll n) {
39     ll ret = 1;
40     while (n > 0) {
41         if (n & 1) ret = mmul(ret, a);
42         a = mmul(a, a);
43         n >>= 1;
44     }
45     return ret;
46 }
47
48 vector<ll> X, iX;
49 vector<int> rev;
50 void init_ntt() {
51     X.assign(maxn, 1); // x1 = g^((p-1)/n)
52     iX.assign(maxn, 1);
53
54     ll u = pw(g, (MOD-1)/maxn);
55     ll iu = pw(u, MOD-2);
56     for (int i = 1; i < maxn; i++) {
57         X[i] = mmul(X[i-1], u);
58         iX[i] = mmul(iX[i-1], iu);
59     }
60
61     if ((int)rev.size() == maxn) return;
62     rev.assign(maxn, 0);
63     for (int i = 1, hb = -1; i < maxn; i++) {
64         if (!(i & (i-1))) hb++;
65         rev[i] = rev[i ^ (1<<hb)] | (1<<(maxk-hb-1));
66     }
67
68 template<typename T>
69 void NTT(vector<T>& a, bool inv=false) {
70     int _n = (int)a.size();
71     int k = __lg(_n) + ((1<<__lg(_n)) != _n);
72     int n = 1<<k;
73     a.resize(n, 0);
74
75     short shift = maxk-k;
76     for (int i = 0; i < n; i++)
77         if (i > (rev[i]>>shift))
78             swap(a[i], a[rev[i]>>shift]);
79     for (int len = 2, half = 1, div = maxn>>1; len <= n; len<=1, half<=1, div>>=1) {
80         for (int i = 0; i < n; i += len) {
81             for (int j = 0; j < half; j++) {
82                 T u = a[i+j];
83                 T v = mmul(a[i+j+half], (inv ? iX[j*div] : X[j*div]));
84                 a[i+j] = (u+v >= MOD ? u+v-MOD : u+v);
85                 a[i+j+half] = (u-v < 0 ? u-v+MOD : u-v);
86             }
87         }
88         if (inv) {
89             T dn = pw(n, MOD-2);
90             for (auto& x : a)
91                 x = mmul(x, dn);
92         }
93     }
94
95 template<typename T>
96 inline void shrink(vector<T>& a) {
97     int cnt = (int)a.size();
98     for (; cnt > 0; cnt--) if (a[cnt-1]) break;
99     a.resize(max(cnt, 1));
100 }
101
102 template<typename T>
103 vector<T>& operator*=(vector<T>& a, vector<T> b) {
104     int na = (int)a.size();

```

```

100 int nb = (int)b.size();
101 a.resize(na + nb - 1, 0);
102 b.resize(na + nb - 1, 0);
103
104 NTT(a); NTT(b);
105 for (int i = 0; i < (int)a.size(); i++)
106     a[i] = mmul(a[i], b[i]);
107 NTT(a, true);
108
109 shrink(a);
110 return a;
111 }
112 inline ll crt(ll a0, ll a1, ll m1, ll m2, ll
    inv_m1_mod_m2){
113     // x ≡ a0 (mod m1), x ≡ a1 (mod m2)
114     // t = (a1 - a0) * inv(m1) mod m2
115     // x = a0 + t * m1 (mod m1*m2)
116     ll t = chmod(a1 - a0);
117     if (t < 0) t += m2;
118     t = (ll)((__int128)t * inv_m1_mod_m2 % m2);
119     return a0 + (ll)((__int128)t * m1);
120 }
121 void mul_crt() {
122     // a copy to a1, a2 | b copy to b1, b2
123     ll M1 = 998244353, M2 = 1004535809;
124     g = 3; set_mod(M1); init_ntt(); a1 *= b1;
125     g = 3, set_mod(M2); init_ntt(); a2 *= b2;
126
127     ll inv_m1_mod_m2 = pw(M1, M2 - 2);
128     for (int i = 2; i <= 2 * k; i++)
129         cout << crt(a1[i], a2[i], M1, M2, inv_m1_mod_m2)
130         << ' ';
131     cout << endl;
132 }
133 /* P = r*2^k + 1
134 P          r    k    g
135 998244353  119  23    3
136 1004535809 479  21    3
137
138 P          r    k    g
139 3          1    1    2
140 5          1    2    2
141 17         1    4    3
142 97         3    5    5
143 193        3    6    5
144 257        1    8    3
145 7681       15    9   17
146 12289      3   12   11
147 40961      5   13    3
148 65537      1   16    3
149 786433     3   18   10
150 5767169    11  19    3
151 7340033    7   20    3
152 23068673   11  21    3
153 104857601  25  22    3
154 167772161  5   25    3
155 469762049  7   26    3
156 1004535809 479 21    3
157 2013265921 15  27   31
158 2281701377 17  27    3
159 3221225473 3  30    5
160 75161927681 35 31    3
161 77309411329 9  33    7
162 206158430209 3  36   22
163 2061584302081 15 37    7
164 2748779069441 5  39    3
165 6597069766657 3  41    5
166 39582418599937 9  42    5
167 79164837199873 9  43    5
168 263882790666241 15 44    7
169 1231453023109121 35 45    3
170 1337006139375617 19 46    3
171 3799912185593857 27 47    5
172 4222124650659841 15 48   19
173 7881299347898369 7  50    6
174 31525197391593473 7  52    3
175 180143985094819841 5  55    6
176 1945555039024054273 27 56    5
177 4179340454199820289 29 57    3
178 9097271247288401921 505 54 6 */

```

12.16 Counting Primes

```

1 // prime_count - #primes in [1..n] (O(n^{2/3}) time,
  O(sqrt(n)) memory)
2
3 using u64 = unsigned long long;
4 static inline u64 prime_count(u64 n){
5     if(n<=1) return 0;

```

```

6 int v = (int)floor(sqrt((long double)n));
7 int s = (v+1)>>1, pc = 0;
8 vector<int> smalls(s), roughs(s), skip(v+1);
9 vector<long long> larges(s);
10
11 for(int i=0; i<s; ++i){
12     smalls[i]=i;
13     roughs[i]=2*i+1;
14     larges[i]=(long long)((n/roughs[i]-1)>>1);
15 }
16
17 for(int p=3; p<=v; p+=2) if(!skip[p]){
18     int q = p*p;
19     if(1LL*q*q > (long long)n) break;
20     skip[p]=1;
21     for(int i=q; i<=v; i+=2*p) skip[i]=1;
22
23     int ns=0;
24     for(int k=0; k<s; ++k){
25         int i = roughs[k];
26         if(skip[i]) continue;
27         u64 d = (u64)i * (u64)p;
28         long long sub = (d <= (u64)v)
29             ? larges[smalls[(int)(d>>1)] - pc]
30             : smalls[(int)((n/d - 1) >> 1)];
31         larges[ns] = larges[k] - sub + pc;
32         roughs[ns++] = i;
33     }
34     s = ns;
35     for(int i=(v-1)>>1, j=((v/p)-1)|1; j>=p; j-=2){
36         int c = smalls[j>>1] - pc;
37         for(int e=(j*p)>>1; i>=e; --i) smalls[i] -= c;
38     }
39     ++pc;
40 }
41
42 larges[0] += 1LL*(s + 2*(pc-1))*(s-1) >> 1;
43 for(int k=1; k<s; ++k) larges[0] -= larges[k];
44
45 for(int l=1; l<s; ++l){
46     int q = roughs[l];
47     u64 m = n / (u64)q;
48     long long t = 0;
49     int e = smalls[(int)((m/q - 1) >> 1)] - pc;
50     if(e < l+1) break;
51     for(int k=l+1; k<=e; ++k) t += smalls[(int)((m/
    (u64)roughs[k] - 1) >> 1)];
52     larges[0] += t - 1LL*(e - l)*(pc + l - 1);
53 }
54 return (u64)(larges[0] + 1);
55 }

```

12.17 Linear Sieve for Other Number Theoretic Functions

```

1 // linear_sieve(n, primes, lp, phi, mu, d, sigma)
2 // Outputs over the index range 0..n (n >= 1):
3 //   primes : all primes in [2..n], increasing.
4 //   lp      : lowest prime factor; lp[1]=0, lp[x] is the
5 //             smallest prime dividing x.
6 //   phi      : Euler totient, phi[x] = |{1<=k<=x :
7 //             gcd(k,x)=1}|. Multiplicative.
8 //   mu       : Möbius; mu[1]=1, mu[x]=0 if x has a squared
9 //             prime factor, else (-1)^{#distinct primes}.
10 //   d        : number of divisors; if x=∏ p_i^{e_i}, then
11 //             d[x]=∏(e_i+1). Multiplicative.
12 //   sigma    : sum of divisors; if x=∏ p_i^{e_i}, then
13 //             sigma[x]=∏(1+p_i+...+p_i^{e_i}). (use ll)
14 // Complexity: O(n) time, O(n) memory.
15 // Notes: Arrays are resized inside; primes is cleared
16 //        and reserved. sigma uses ll to avoid 32-bit overflow.
17
18 static inline void linear_sieve(
19     int n,
20     std::vector<int> &primes,
21     std::vector<int> &lp,
22     std::vector<int> &phi,
23     std::vector<int> &mu,
24     std::vector<int> &d,
25     std::vector<ll> &sigma
26 ) {
27     lp.assign(n + 1, 0); phi.assign(n + 1, 0); mu.assign(n
28     + 1, 0); d.assign(n + 1, 0); sigma.assign(n + 1, 0);
29     primes.clear(); primes.reserve(n > 1 ? n / 10 : 0);
30     std::vector<int> cnt(n + 1, 0), core(n + 1, 1);
31     std::vector<ll> p_pow(n + 1, 1), sum_p(n + 1, 1);
32     phi[1] = mu[1] = d[1] = sigma[1] = 1;
33
34     for (int i = 2; i <= n; ++i) {
35         if (!lp[i]) {
36             lp[i] = i; primes.push_back(i);

```



```

31     phi[i] = i - 1; mu[i] = -1; d[i] = 2;
32     cnt[i] = 1; p_pow[i] = i; core[i] = 1;
33     sum_p[i] = 1 + (ll)i; sigma[i] = sum_p[i];
34 }
35 for (int p : primes) {
36     long long ip = 1LL * i * p;
37     if (ip > n) break;
38     lp[ip] = p;
39     if (p == lp[ip]) {
40         cnt[ip] = cnt[i] + 1; p_pow[ip] = p_pow[i] * p;
41         core[ip] = core[i];
42         sum_p[ip] = sum_p[i] + p_pow[ip];
43         phi[ip] = phi[i] * p; mu[ip] = 0;
44         d[ip] = d[core[ip]] * (cnt[ip] + 1);
45         sigma[ip] = sigma[core[ip]] * sum_p[ip];
46         break; // critical for linear complexity
47     } else {
48         cnt[ip] = 1; p_pow[ip] = p; core[ip] = i;
49         sum_p[ip] = 1 + (ll)p;
50         phi[ip] = phi[i] * (p - 1); mu[ip] = -mu[i];
51         d[ip] = d[i] * 2;
52         sigma[ip] = sigma[i] * sum_p[ip];
53     }
54 }
55 }
56
57 // Optional helper: factorize x in O(log x) using lp
58 // (requires x in [2..n])
59 static inline std::vector<std::pair<int,int>>
60 factorize(int x, const std::vector<int>& lp) {
61     std::vector<std::pair<int,int>> res;
62     while (x > 1) {
63         int p = lp[x], e = 0;
64         do { x /= p; ++e; } while (x % p == 0);
65         res.push_back({p, e});
66     }
67     return res;
68 }

```

13 Numerical

13.1 Polynomials and recurrences

```

1 /* Author: David Rydh, Per Austrin
2 * Description: */
3 #pragma once
4 struct Poly {
5     vector<double> a;
6     double operator()(double x) const {
7         double val = 0;
8         for (int i = sz(a); i--;) (val *= x) += a[i];
9         return val;
10    }
11    void diff() {
12        rep(i, 1, sz(a)) a[i-1] = i*a[i];
13        a.pop_back();
14    }
15    void divroot(double x0) {
16        double b = a.back(), c; a.back() = 0;
17        for (int i=sz(a)-1; i--;) c = a[i], a[i] = a[i+1]*x0+b,
18        b=c;
19        a.pop_back();
20    };
21
22    /* Author: Per Austrin (License: CC0)
23    * Description: Finds the real roots to a polynomial.
24    * Usage: polyRoots({{2,-3,1}},-1e9,1e9) // solve x^2-3x+2 =
25    * 0
26    * Time: O(n^2 \log(1/|epsilon|)) */
27    #include "Polynomial.h"
28    vector<double> polyRoots(Poly p, double xmin, double xmax) {
29        if (sz(p.a) == 2) { return {-p.a[0]/p.a[1]}; }
30        vector<double> ret;
31        Poly der = p;
32        der.diff();
33        auto dr = polyRoots(der, xmin, xmax);
34        dr.push_back(xmin-1);
35        dr.push_back(xmax+1);
36        sort(all(dr));
37        rep(i, 0, sz(dr)-1) {
38            double l = dr[i], h = dr[i+1];
39            bool sign = p(l) > 0;
40            if (sign ^ (p(h) > 0)) {
41                rep(it, 0, 60) { // while (h - l > 1e-8)
42                    double m = (l + h) / 2, f = p(m);
43                    if ((f <= 0) ^ sign) l = m;
44                    else h = m;
45                }
46                ret.push_back((l + h) / 2);
47            }
48        }
49    }

```

```

25     }
26 }
27 return ret;
28 }
29
30 /* Author: Simon Lindholm (License: CC0)
31 * Description: Given $n$ points $(x[i], y[i])$, computes an
32 $n-1$-degree polynomial $p$ that
33 * passes through them: $p(x) = a[0]*x^0 + \dots +
34 a[n-1]*x^{n-1}$.
35 * For numerical precision, pick $x[k] = c*\cos(k/(n-1)*\pi)$,
36 $k=0 \dots n-1$.
37 * Time: $O(n^2)$ */
38 #pragma once
39 typedef vector<double> vd;
40 vd interpolate(vd x, vd y, int n) {
41     vd res(n), temp(n);
42     rep(k, 0, n-1) rep(i, k+1, n)
43         y[i] = (y[i] - y[k]) / (x[i] - x[k]);
44     double last = 0; temp[0] = 1;
45     rep(k, 0, n) rep(i, 0, n) {
46         res[i] += y[k] * temp[i];
47         swap(last, temp[i]);
48         temp[i] -= last * x[k];
49     }
50     return res;
51 }
52
53 /* Author: koukanni (License: CC0)
54 * Description: Lagrange interpolation for polynomial
55 evaluation at integer points.
56 * Given polynomial values $f[0], f[1], \dots, f[n-1]$, computes
57 $f(c)$.
58 * Time: $O(n)$
59 * Status: tested */
60 Mint lagrange_iota(const vector<Mint> &f, const int c) {
61     const int n = int(size(f));
62     if (c < n) {
63         return f[c];
64     }
65     vector<Mint> fac(n), ifac(n);
66     fac[0] = 1;
67     for (int i = 1; i < n; i++) fac[i] = fac[i-1] * i;
68     ifac[n-1] = fac[n-1].inv();
69     for (int i = n-1; i >= 1; i--) ifac[i-1] = ifac[i] *
70     i;
71     vector<Mint> g(n);
72     for (int i = 0; i < n; i++) {
73         g[i] = f[i] * ifac[i] * ifac[n-1-i];
74         if ((n-1-i) & 1) g[i] *= -1;
75     }
76     vector<Mint> lp(n+1), rp(n+1);
77     lp[0] = rp[n] = 1;
78     for (int i = 0; i < n; i++) lp[i+1] = lp[i] * Mint(c -
79     i);
80     for (int i = n-1; i >= 0; i--) rp[i] = rp[i+1] *
81     Mint(c - i);
82     Mint ans = 0;
83     for (int i = 0; i < n; i++) ans += g[i] * lp[i] * rp[i +
84     1];
85     return ans;
86 }
87
88 /* Author: Lucian Bicsi (License: CC0)
89 * Description: Recovers any $n$-order linear recurrence
90 relation from the first
91 * $2n$ terms of the recurrence.
92 * Useful for guessing linear recurrences after brute-forcing
93 the first terms.
94 * Should work on any field, but numerical stability for
95 floats is not guaranteed.
96 * Output will have size $\le n$.
97 * Usage: berlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2}
98 * Time: $O(n^2)$, Status: brute-force-tested mod 5 for $n \le 5$
99 and all $s$ */
100 #include "../number-theory/ModPow.h"
101 vector<ll> berlekampMassey(vector<ll> s) {
102     int n = sz(s), L = 0, m = 0;
103     vector<ll> C(n), B(n), T;
104     C[0] = B[0] = 1;
105     ll b = 1;
106     rep(i, 0, n) { ++m;
107         ll d = s[i] % mod;
108         rep(j, 1, L+1) d = (d + C[j] * s[i-j]) % mod;
109         if (!d) continue;
110         T = C; ll coef = d * modpow(b, mod-2) % mod;
111         rep(j, m, n) C[j] = (C[j] - coef * B[j-m]) % mod;
112         if (2 * L > i) continue;
113         L = i + 1 - L; B = T; b = d; m = 0;
114     }
115     // assert(2 * L + 30 <= n);
116 }

```

```

25 C.resize(L + 1); C.erase(C.begin());
26 for (ll& x : C) x = (mod - x) % mod;
27 return C;
28 }

/* Author: Lucian Bicsi (License: CC0)
2* Description: Generates the $k$'th term of an $n$-order
3* linear recurrence $S[i] = \sum_j S[i-j-1]tr[j]$,
4* given $S[0 \dots \text{ldots } \text{ge } n-1]$ and $tr[0 \dots n-1]$.
5* Faster than matrix multiplication.
6* Useful together with Berlekamp-Massey.
7* Usage: linearRec({0, 1}, {1, 1}, k) // k'th Fibonacci
   number
8* Time: $O(n^2 \log k)$
9* Status: brute-force-tested mod 5 for $n \le 5$ */
10 const ll mod = 5; /** exclude-line */
11 typedef vector<ll> Poly;
12 ll linearRec(Poly S, Poly tr, ll k) {
13     int n = sz(tr);
14     auto combine = [&](Poly a, Poly b) {
15         Poly res(n + 2, 1);
16         rep(i, 0, n+1) rep(j, 0, n+1)
17             res[i + j] = (res[i + j] + a[i] * b[j]) % mod;
18         for (int i = 2 * n; i > n; --i) rep(j, 0, n)
19             res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) %
mod;
20         res.resize(n + 1);
21         return res;
22     };
23     Poly pol(n + 1, e(pol));
24     pol[0] = e[1] = 1;
25     for (++k; k; k /= 2) {
26         if (k % 2) pol = combine(pol, e);
27         e = combine(e, e);
28     }
29     ll res = 0;
30     rep(i, 0, n) res = (res + pol[i + 1] * S[i]) % mod;
31     return res;
32 }

```

13.2 Optimization

```

/* Author: Ulf Lundstrom (License: CC0)
2* Description: Finds the argument minimizing the function
   $f$ in the interval $[a,b]$
3* assuming $f$ is unimodal on the interval, i.e. has only
   one local minimum and no local
4* maximum. The maximum error in the result is $eps$. Works
   equally well for maximization
5* with a small change in the code. See TernarySearch.h in
   the Various chapter for a
6* discrete version.
7* Usage: double func(double x) { return 4x+.3*x*x; }
8* double xmin = gss(-1000,1000,func);
9* Time: $O(\log((b-a) / \epsilon))$, Status: tested */
10 // It is important for r to be precise, otherwise we don't
   necessarily maintain the inequality $a < x_1 < x_2 < b$.
11 double gss(double a, double b, double (*f)(double)) {
12     double r = (sqrt(5)-1)/2, eps = 1e-7;
13     double x1 = b - r*(b-a), x2 = a + r*(b-a);
14     double f1 = f(x1), f2 = f(x2);
15     while (b-a > eps)
16         if (f1 < f2) { //change to > to find maximum
17             b = x2; x2 = x1; f2 = f1;
18             x1 = b - r*(b-a); f1 = f(x1);
19         } else {
20             a = x1; x1 = x2; f1 = f2;
21             x2 = a + r*(b-a); f2 = f(x2);
22         }
23     return a;
24 }

/* Author: Simon Lindholm (License: CC0)
2* Description: Poor man's optimization for unimodal
   functions.
3* Status: used with great success */
4 typedef array<double, 2> P;
5 template<class F> pair<double, P> hillClimb(P start, F f) {
6     pair<double, P> cur(f(start), start);
7     for (double jmp = 1e9; jmp > 1e-20; jmp /= 2) {
8         rep(j, 0, 100) rep(dx, -1, 2) rep(dy, -1, 2) {
9             P p = cur.second;
10            p[0] += dx*jmp;
11            p[1] += dy*jmp;
12            cur = min(cur, make_pair(f(p), p));
13        }
14    }
15    return cur;
16 }

/* Author: Simon Lindholm (License: CC0)

```

```

2* Description: Simple integration of a function over an
   interval using
3* Simpson's rule. The error should be proportional to $h^4$,
   although in
4* practice you will want to verify that the result is stable
   to desired
5* precision when epsilon changes.
6* Status: mostly untested */
7 template<class F>
8 double quad(double a, double b, F f, const int n = 1000) {
9     double h = (b - a) / 2 / n, v = f(a) + f(b);
10    rep(i, 1, n*2)
11        v += f(a + i*h) * (i&1 ? 4 : 2);
12    return v * h / 3;
13 }

/* Author: Simon Lindholm (License: CC0)
2* Description: Fast integration using an adaptive Simpson's
   rule.
3* Usage:
4* double sphereVolume = quad(-1, 1, [](double x) {
5*     return quad(-1, 1, [&](double y) {
6*         return quad(-1, 1, [&](double z) {
7*             return x*x + y*y + z*z < 1; });});});
8* Status: mostly untested */
9 typedef double d;
10 #define S(a,b) (f(a) + 4*f((a+b) / 2) + f(b)) * (b-a) / 6
11 template <class F>
12 d rec(F& f, d a, d b, d eps, d S) {
13     d c = (a + b) / 2;
14     d S1 = S(a, c), S2 = S(c, b), T = S1 + S2;
15     if (abs(T - S) <= 15 * eps || b - a < 1e-10)
16         return T + (T - S) / 15;
17     return rec(f, a, c, eps / 2, S1) + rec(f, c, b, eps / 2,
S2);
18 }
19 template<class F>
20 d quad(d a, d b, F f, d eps = 1e-8) {
21     return rec(f, a, b, eps, S(a, b));
22 }

/* Author: Stanford (License: MIT)
2* Description: Solves a general linear maximization problem:
   maximize $c^T x$ subject to $Ax \le b$, $x \ge 0$.
3* Returns -inf if there is no solution, inf if there are
   arbitrarily good solutions, or the maximum value of $c^T x$
   otherwise.
4* The input vector is set to an optimal $x$ (or in the
   unbounded case, an arbitrary solution fulfilling the
   constraints).
5* Numerical stability is not guaranteed. For better
   performance, define variables such that $x = 0$ is viable.
6* Usage:
7* vvd A = {{1,-1}, {-1,1}, {-1,-2}};
8* vd b = {1,1,-4}, c = {-1,-1}, x;
9* T val = LPSolver(A, b, c).solve(x);
10* Time: $O(NM * \log \text{pivots})$, where a pivot may be e.g. an edge
   relaxation. $O(2^n)$ in the general case.
11* Status: seems to work? */
12 typedef double T; // long double, Rational, double +
   mod<P>...
13 typedef vector<T> vd;
14 typedef vector<vd> vvd;
15 const T eps = 1e-8, inf = 1/.0;
16 #define MP make_pair
17 #define ltj(X) if(s == -1 || MP(X[j],N[j]) < MP(X[s],N[s]))
   s=j
18 struct LPSolver {
19     int m, n; vi N, B; vvd D;
20     LPSolver(const vvd& A, const vd& b, const vd& c) :
21         m(sz(b)), n(sz(c)), N(n+1), B(m), D(m+2, vd(n+2)) {
22         rep(i, 0, m) rep(j, 0, n) D[i][j] = A[i][j];
23         rep(i, 0, m) { B[i] = n+i; D[i][n] = -1; D[i][n+1] =
b[i]; }
24         rep(j, 0, n) { N[j] = j; D[m][j] = -c[j]; }
25         N[n] = -1; D[m+1][n] = 1;
26     }
27     void pivot(int r, int s) {
28         T *a = D[r].data(), inv = 1 / a[s];
29         rep(i, 0, m+2) if (i != r && abs(D[i][s]) > eps) {
30             T *b = D[i].data(), inv2 = b[s] * inv;
31             rep(j, 0, n+2) b[j] -= a[j] * inv2;
32             b[s] = a[s] * inv2;
33         }
34         rep(j, 0, n+2) if (j != s) D[r][j] *= inv;
35         rep(i, 0, m+2) if (i != r) D[i][s] *= -inv;
36         D[r][s] = inv;
37         swap(B[r], N[s]);
38     }
39     bool simplex(int phase) {
40         int x = m + phase - 1;

```

```

41 for (;;) {
42     int s = -1;
43     rep(j,0,n+1) if (N[j] != -phase) ltj(D[x]);
44     if (D[x][s] >= -eps) return true;
45     int r = -1;
46     rep(i,0,m) {
47         if (D[i][s] <= eps) continue;
48         if (r == -1 || MP(D[i][n+1] / D[i][s], B[i])
49             < MP(D[r][n+1] / D[r][s], B[r])) r = i;
50     }
51     if (r == -1) return false;
52     pivot(r, s);
53 }
54 }
55 T solve(vd &x) {
56     int r = 0;
57     rep(i,1,m) if (D[i][n+1] < D[r][n+1]) r = i;
58     if (D[r][n+1] < -eps) {
59         pivot(r, n);
60         if (!simplex(2) || D[m+1][n+1] < -eps) return -inf;
61         rep(i,0,m) if (B[i] == -1) {
62             int s = 0;
63             rep(j,1,n+1) ltj(D[i]);
64             pivot(i, s);
65         }
66     }
67     bool ok = simplex(1); x = vd(n);
68     rep(i,0,m) if (B[i] < n) x[B[i]] = D[i][n+1];
69     return ok ? D[m][n+1] : inf;
70 }
71 };

```

13.3 Matrices

```

1/* Author: Simon Lindholm (License: CC0)
2* Description: Calculates determinant of a matrix. Destroys
the matrix.
3* Time:  $O(N^3)$ , Status: somewhat tested */
4double det(vector<vector<double>>& a) {
5    int n = sz(a); double res = 1;
6    rep(i,0,n) {
7        int b = i;
8        rep(j,i+1,n) if (fabs(a[j][i]) > fabs(a[b][i])) b = j;
9        if (i != b) swap(a[i], a[b]), res *= -1;
10       res *= a[i][i];
11       if (res == 0) return 0;
12       rep(j,i+1,n) {
13           double v = a[j][i] / a[i][i];
14           if (v != 0) rep(k,i+1,n) a[j][k] -= v * a[i][k];
15       }
16   }
17   return res;
18}

1/* Author: Unknown
2* Description: Calculates determinant using modular
arithmetic.
3* Modulos can also be removed to get a pure-integer version.
4* Time:  $O(N^3)$ 
5* Status: brute-force-tested for  $N \leq 3$ , mod  $\leq 7$  */
6const ll mod = 12345;
7ll det(vector<vector<ll>>& a) {
8    int n = sz(a); ll ans = 1;
9    rep(i,0,n) {
10        rep(j,i+1,n) {
11            while (a[j][i] != 0) { // gcd step
12                ll t = a[i][i] / a[j][i];
13                if (t) rep(k,i,n)
14                    a[i][k] = (a[i][k] - a[j][k] * t) % mod;
15                swap(a[i], a[j]);
16                ans *= -1;
17            }
18        }
19        ans = ans * a[i][i] % mod;
20        if (!ans) return 0;
21    }
22    return (ans + mod) % mod;
23}

1/* Author: Per Austrin, Simon Lindholm (License: CC0)
2* Description: Solves  $A * x = b$ . If there are multiple
solutions, an arbitrary one is returned.
3* Returns rank, or -1 if no solutions. Data in  $A$  and  $b$ 
is lost.
4* Time:  $O(n^2 m)$ 
5* Status: tested on kattis:equationsolver, and brute-force-
tested mod 3 and 5 for  $n, m \leq 3$  */
6typedef vector<double> vd;
7const double eps = 1e-12;
8int solveLinear(vector<vd>& A, vd& b, vd& x) {
9    int n = sz(A), m = sz(x), rank = 0, br, bc;

```

```

10 if (n) assert(sz(A[0]) == m);
11 vi col(m); iota(all(col), 0);
12 rep(i,0,n) {
13     double v, bv = 0;
14     rep(r,i,n) rep(c,i,m)
15         if ((v = fabs(A[r][c])) > bv)
16             br = r, bc = c, bv = v;
17     if (bv <= eps) {
18         rep(j,i,n) if (fabs(b[j]) > eps) return -1;
19         break;
20     }
21     swap(A[i], A[br]);
22     swap(b[i], b[br]);
23     swap(col[i], col[bc]);
24     rep(j,0,n) swap(A[j][i], A[j][bc]);
25     bv = 1/A[i][i];
26     rep(j,i+1,n) {
27         double fac = A[j][i] * bv;
28         b[j] -= fac * b[i];
29         rep(k,i+1,m) A[j][k] -= fac*A[i][k];
30     }
31     rank++;
32 }
33 x.assign(m, 0);
34 for (int i = rank; i--;) {
35     b[i] /= A[i][i];
36     x[col[i]] = b[i];
37     rep(j,0,i) b[j] -= A[j][i] * b[i];
38 }
39 return rank; // (multiple solutions if rank < m)
40}

```

```

1/* Author: Simon Lindholm (License: CC0)
2* Description: To get all uniquely determined values of  $x$ 
back from SolveLinear, make the following changes:
3* Status: tested on kattis:equationsolverplus, stress-tested
*/
4#include "SolveLinear.h"
5rep(j,0,n) if (j != i) // instead of rep(j,i+1,n)
6// ... then at the end:
7x.assign(m, undefined);
8rep(i,0,rank) {
9    rep(j,rank,m) if (fabs(A[i][j]) > eps) goto fail;
10   x[col[i]] = b[i] / A[i][i];
11   fail; }

1/* Author: Simon Lindholm (License: CC0)
2* Description: Solves  $AX = b$  over  $\mathbb{F}_2$ . If there
are multiple solutions, one is returned arbitrarily.
3* Returns rank, or -1 if no solutions. Destroys  $A$  and  $b$ .
4* Time:  $O(n^2 m)$ 
5* Status: brute-force-tested for  $n, m \leq 4$  */
6typedef bitset<1000> bs;
7int solvelinear(vector<bs>& A, vi& b, bs& x, int m) {
8    int n = sz(A), rank = 0, br;
9    assert(m <= sz(x));
10   vi col(m); iota(all(col), 0);
11   rep(i,0,n) {
12       for (br=i; br<n; ++br) if (A[br].any()) break;
13       if (br == n) {
14           rep(j,i,n) if (b[j]) return -1;
15           break;
16       }
17       int bc = (int)A[br]._Find_next(i-1);
18       swap(A[i], A[br]);
19       swap(b[i], b[br]);
20       swap(col[i], col[bc]);
21       rep(j,0,n) if (A[j][i] != A[j][bc]) {
22           A[j].flip(i); A[j].flip(bc);
23       }
24       rep(j,i+1,n) if (A[j][i]) {
25           b[j] ^= b[i];
26           A[j] ^= A[i];
27       }
28       rank++;
29   }
30   x = bs();
31   for (int i = rank; i--;) {
32       if (!b[i]) continue;
33       x[col[i]] = 1;
34       rep(j,0,i) b[j] ^= A[j][i];
35   }
36   return rank; // (multiple solutions if rank < m)
37}

1/* Author: Max Bennedich
2* Description: Invert matrix  $A$ . Returns rank; result is
stored in  $A$  unless singular (rank < n).
3* Can easily be extended to prime moduli; for prime powers,
repeatedly

```

```

4 * set  $A^{-1} = A^{-1} (2I - AA^{-1}) \pmod{p^k}$ 
   where  $A^{-1}$  starts as
5 * the inverse of  $A \pmod{p}$ , and  $k$  is doubled in each step.
6 * Time:  $O(n^3)$ 
7 * Status: Slightly tested */
8 int matInv(vector<vector<double>>& A) {
9     int n = sz(A); vi col(n);
10    vector<vector<double>> tmp(n, vector<double>(n));
11    rep(i,0,n) tmp[i][i] = 1, col[i] = i;
12    rep(i,0,n) {
13        int r = i, c = i;
14        rep(j,i,n) rep(k,i,n)
15            if (fabs(A[j][k]) > fabs(A[r][c]))
16                r = j, c = k;
17        if (fabs(A[r][c]) < 1e-12) return i;
18        A[i].swap(A[r]); tmp[i].swap(tmp[r]);
19        rep(j,0,n)
20            swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
21        swap(col[i], col[c]);
22        double v = A[i][i];
23        rep(j,i+1,n) {
24            double f = A[j][i] / v;
25            A[j][i] = 0;
26            rep(k,i+1,n) A[j][k] -= f*A[i][k];
27            rep(k,0,n) tmp[j][k] -= f*tmp[i][k];
28        }
29        rep(j,i+1,n) A[i][j] /= v;
30        rep(j,0,n) tmp[i][j] /= v;
31        A[i][i] = 1;
32    }
33    /// forget A at this point, just eliminate tmp backward
34    for (int i = n-1; i > 0; --i) rep(j,0,i) {
35        double v = A[j][i];
36        rep(k,0,n) tmp[j][k] -= v*tmp[i][k];
37    }
38    rep(i,0,n) rep(j,0,n) A[col[i]][col[j]] = tmp[i][j];
39    return n;
40 }

1/* Author: Simon Lindholm
2 * Description: Invert matrix  $A$  modulo a prime.
3 * Returns rank; result is stored in  $A$  unless singular
   (rank < n).
4 * For prime powers, repeatedly set  $A^{-1} = A^{-1} (2I -$ 
    $AA^{-1}) \pmod{p^k}$  where  $A^{-1}$  starts as
5 * the inverse of  $A \pmod{p}$ , and  $k$  is doubled in each step.
6 * Time:  $O(n^3)$ 
7 * Status: Slightly tested */
8 #include "number-theory/ModPow.h"
9 int matInv(vector<vector<ll>>& A) {
10    int n = sz(A); vi col(n);
11    vector<vector<ll>> tmp(n, vector<ll>(n));
12    rep(i,0,n) tmp[i][i] = 1, col[i] = i;
13    rep(i,0,n) {
14        int r = i, c = i;
15        rep(j,i,n) rep(k,i,n) if (A[j][k]) {
16            r = j; c = k; goto found;
17        }
18        return i;
19    found:
20    A[i].swap(A[r]); tmp[i].swap(tmp[r]);
21    rep(j,0,n)
22        swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
23    swap(col[i], col[c]);
24    ll v = modpow(A[i][i], mod - 2);
25    rep(j,i+1,n) {
26        ll f = A[j][i] * v % mod;
27        A[j][i] = 0;
28        rep(k,i+1,n) A[j][k] = (A[j][k] - f*A[i][k]) % mod;
29        rep(k,0,n) tmp[j][k] = (tmp[j][k] - f*tmp[i][k]) %
mod;
30    }
31    rep(j,i+1,n) A[i][j] = A[i][j] * v % mod;
32    rep(j,0,n) tmp[i][j] = tmp[i][j] * v % mod;
33    A[i][i] = 1;
34    }
35    for (int i = n-1; i > 0; --i) rep(j,0,i) {
36        ll v = A[j][i];
37        rep(k,0,n) tmp[j][k] = (tmp[j][k] - v*tmp[i][k]) % mod;
38    }
39    rep(i,0,n) rep(j,0,n)
40        A[col[i]][col[j]] = tmp[i][j] % mod + (tmp[i][j] <
0)*mod;
41    return n;
42 }

1/* Author: Ulf Lundstrom, Simon Lindholm (License: CC0)
2 * Description: Solves the tridiagonal matrix equation
   system:
3 *  $\begin{bmatrix} b_0 & & & & \\ & d_0 & p_0 & & \\ & & & \ddots & \\ & & & & d_{n-2} & p_{n-2} \\ & & & & & d_{n-1} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{n-1} \end{bmatrix}$ 

```

```

4 *  $\begin{bmatrix} b_1 & & & & \\ & d_1 & p_1 & & \\ & & d_2 & p_2 & \\ & & & \ddots & \\ & & & & d_{n-1} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{n-1} \end{bmatrix}$ 
5 *  $\begin{bmatrix} b_2 & & & & \\ & d_2 & p_2 & & \\ & & d_3 & p_3 & \\ & & & \ddots & \\ & & & & d_n \end{bmatrix} \begin{bmatrix} x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix}$ 
6 *  $\begin{bmatrix} \dots & & & & \\ & \dots & \dots & \dots & \dots \\ & & & & \dots \end{bmatrix} \begin{bmatrix} \dots \\ \dots \\ \dots \end{bmatrix}$ 
7 *  $\begin{bmatrix} b_{n-1} & & & & \\ & d_{n-1} & p_{n-1} & & \\ & & & \ddots & \\ & & & & d_n \end{bmatrix} \begin{bmatrix} x_{n-1} \\ x_n \end{bmatrix}$ 
8 * Application:
9 * Solves linear recurrence systems of the form:
10 *  $a_i = b_i * a_{i-1} + c_i * a_{i+1} + d_i$  ( $1 \leq i \leq n$ )
   where  $a_0, a_{n+1}, b_i, c_i$ , and  $d_i$  are given.
12 * The solution vector  $a$  can be obtained via:
13 *  $d = \{1, -1, -1, \dots, -1, 1\}$ 
14 *  $p = \{0, c_1, c_2, \dots, c_n\}$ 
15 *  $q = \{b_1, b_2, \dots, b_n, 0\}$ 
16 *  $b = \{a_0, d_1, d_2, \dots, d_n, a_{n+1}\}$ 
17 * Numerical Stability:
18 * - Fails if the solution is not unique.
19 * - Numerically stable without pivot checks if:
20 * 1) Strictly diagonally dominant:
21 *  $|d_i| > |p_i| + |q_{i-1}|$  for all  $i$ , OR
22 *  $|d_i| > |p_{i-1}| + |q_i|$  for all  $i$ 
23 * 2) The matrix is symmetric positive-definite.
24 * Complexity:  $O(N)$  time,  $O(N)$  space. Status: Brute-force
   tested mod 5 and 7, stress-tested for real matrices. */
25 typedef double T;
26 vector<T> tridiagonal(vector<T> diag, const vector<T>&
super,
27 const vector<T>& sub, vector<T> b) {
28     int n = sz(b); vi tr(n);
29     rep(i,0,n-1) {
30         if (abs(diag[i]) < 1e-9 * abs(super[i])) { // diag[i] ==
0
31             b[i+1] -= b[i] * diag[i+1] / super[i];
32             if (i+2 < n) b[i+2] -= b[i] * sub[i+1] / super[i];
33             diag[i+1] = sub[i]; tr[i+1] = 1;
34         } else {
35             diag[i+1] -= super[i]*sub[i]/diag[i];
36             b[i+1] -= b[i]*sub[i]/diag[i];
37         }
38     }
39     for (int i = n; i--;) {
40         if (tr[i]) {
41             swap(b[i], b[i-1]);
42             diag[i-1] = diag[i];
43             b[i] /= super[i-1];
44         } else {
45             b[i] /= diag[i];
46             if (i) b[i-1] -= b[i]*super[i-1];
47         }
48     }
49     return b;
50 }

```

13.4 Fourier transforms

```

1/**
2 * Author: chilli, Ludo Pulles, Simon Lindholm (License:
   CC0)
3 * Description: Higher precision FFT, can be used for
   convolutions modulo arbitrary integers
4 * as long as  $N \log_2 N * \text{mod} < 8.6e14$  (in practice  $10^{16}$  or
   higher).
5 * Inputs must be in  $[0, \text{mod})$ .
6 * Time:  $O(N \log N)$ , where  $N = |A| + |B|$  (twice as slow as
   NTT or normal FFT), Status: stress-tested */
7
8 typedef complex<double> C;
9 typedef vector<ll> vl;
10 void fft(vector<C>& a) {
11     int n = sz(a), L = 31 - __builtin_clz(n);
12     static vector<complex<long double>> R(2, 1);
13     static vector<C> rt(2, 1);
14     for (static int k = 2; k < n; k *= 2) {
15         R.resize(n); rt.resize(n);
16         auto x = polar(1.0L, acos(-1.0L) / k);
17         rep(i, k, 2 * k) rt[i] = R[i] = i & 1 ? R[i / 2] * x :
R[i / 2];
18     }
19     vi rev(n);
20     rep(i, 0, n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
21     rep(i, 0, n) if (i < rev[i]) swap(a[i], a[rev[i]]);
22     for (int k = 1; k < n; k *= 2)
23         for (int i = 0; i < n; i += 2 * k) rep(j, 0, k) {
24             auto x = (double *)&rt[j + k], y = (double *)&a[i + j
+ k];
25             C z(x[0] * y[0] - x[1] * y[1], x[0] * y[1] + x[1] *
y[0]);
26             a[i + j + k] = a[i + j] - z;
27             a[i + j] += z;
28         }
29 }
30 template<int M> vl convMod(const vl &a, const vl &b) {
31     if (a.empty() || b.empty()) return {};

```


14 Combinatorics

14.1 Catalan Number

14.1 Catalan Number

0	1	1	2	5
4	14	42	132	429
8	1430	4862	16796	58786
12	208012	742900	2674440	9694845

- A always $> B$: $C(p+q, p) - 2C(p+q-1, p)$
- A always $\geq B$: $C(p+q, p) \times \frac{p+1-q}{p+1}$

Let X be the original set.
 Let G be the group of operations acting on X .
 Let X^g be the set of x not affected by g .
 Let X/G be the set of orbits. Then the following equation holds:

15 Special Numbers

15.1 Fibonacci Series

15.1 Fibonacci Series

25	75025	121393	196418	317811
29	514229	832040	1346269	2178309
33	3524578	5702887	9227465	14930352

15.2 Prime Numbers

15.2 Prime Numbers

- $\pi(n) \equiv \text{Number of primes } \leq n \approx \frac{n}{(\ln n)-1}$
 $\pi(100) = 25, \pi(200) = 46$
 $\pi(500) = 95, \pi(1000) = 168$
 $\pi(2000) = 303, \pi(4000) = 550$
 $\pi(10^4) = 1229, \pi(10^5) = 9592$
 $\pi(10^6) = 78498, \pi(10^7) = 664579$

- If $n = \prod p_i^{a_i}$, then $\tau(n) = \prod (a_i + 1)$
- Maximum $\tau(n)$ for $n \leq 10^k$:

- Useful bounds: $n \leq 10^6 \Rightarrow \tau(n) \leq 240$
 $n \leq 10^9 \Rightarrow \tau(n) \leq 1344$
 $n \leq 10^{12} \Rightarrow \tau(n) \leq 6720$
 $n \leq 10^{15} \Rightarrow \tau(n) \leq 26880$
 $n \leq 10^{18} \Rightarrow \tau(n) \leq 103680$

- Let $\omega(n)$ = number of distinct prime factors.
- Minimum number with k distinct prime factors is the product of the first k primes.
 - $2 \cdot 3 \cdot 5 \cdot 7 \cdot 11 \cdot 13 \cdot 17 \cdot 19 = 9699690$
 - $2 \cdot 3 \cdot \dots \cdot 23 = 223092870$
 - $2 \cdot 3 \cdot \dots \cdot 29 = 6469693230$
 - $n \leq 10^9 \Rightarrow \omega(n) \leq 9, n \leq 10^{18} \Rightarrow \omega(n) \leq 15$
- Number of square-free divisors = $2^{\omega(n)}$
 - $n \leq 10^9 \Rightarrow \leq 512, n \leq 10^{18} \Rightarrow \leq 32768$